KodeKloud 50 Days Azure Cloud Challenge

Table of Content

Day 01: Create SSH Key Pair for Azure Virtual Machine **6**
Task Description: 6
Solution: 6
Day 02: Create an Azure Virtual Machine **8**
Task Description: 8
Solution: 8
Day 03: Create VM using Azure CLI **10**
Task Description: 10
Solution: 10
Day 04: Create a Virtual Network (VNet) in Azure **12**
Task Description: 12
Solution: 12
Day 05: Create a Virtual Network (IPv4) in Azure **14**
Task Description: 14
Solution: 14
Day 06: Create a Subnet in Azure Virtual Network **16**
Task Description: 16
Solution: 16
Day 07: Create a Public IP Address for Azure VM **18**
Task Description: 18
Solution: 18
Day 08: Attach Managed Disk to Azure Virtual Machine **20**
Task Description: 20
Solution: 20
Day 09: Attach Network Interface Card (NIC) to Azure Virtual Machine **22**
Task Description: 22
Solution: 22
Day 10: Attach Public IP to Azure Virtual Machine **24**
Task Description: 24
Solution: 24
Day 11: Change Azure Virtual Machine Size Using Console **26**
Task Description: 26
Solution: 26
Day 12: Add and Manage Tags for Azure Virtual Machines **27**
Task Description: 27
Solution: 27
Day 13: SSH into an Azure Virtual Machine **29**
Task Description: 29
Solution: 29

Day 14: Create and Attach Managed Disks in Azure **32**
Task Description: 32
Solution: 32
Day 15: Create and Configure Network Security Group (NSG) in Azure **34**
Task Description: 34
Solution: 34
Day 16: Create a Private Azure Blob Storage Container **36**
Task Description: 36
Solution: 36
Day 17: Create a Public Azure Blob Storage Container **38**
Task Description: 38
Solution: 38
Day 18: Copy Data to an Azure Blob Storage Container **40**
Task Description: 40
Solution: 40
Day 19: Convert Public Azure Blob Container to Private **42**
Task Description: 42
Solution: 42
Day 20: Deploy Azure Resources Using ARM Template **44**
Task Description: 44
Solution: 44
Day 21: Assigning Public IP to Virtual Machines **47**
Task Description: 47
Solution: 47
Day 22: Configuring Instances with User Data **49**
Task Description: 49
Solution: 49
Day 23: Automating User Data Configuration Using the CLI **51**
Task Description: 51
Solution: 51
Day 24: Securing Virtual Machine SSH Access **54**
Task Description: 54
Solution: 54
Day 25: Expanding and Managing Disk Storage **56**
Task Description: 56
Solution: 56
Day 26: Deploying Virtual Machines in a Public Virtual Network **59**
Task Description: 59
Solution: 59
Day 27: Deploying Virtual Machines in a Private Virtual Network **62**
Task Description: 62

Solution: 62
Day 28: Troubleshooting Public Virtual Network Configurations 65
Task Description: 65
Solution: 65
Day 29: Working with Azure Container Registry (ACR) 68
Task Description: 68
Solution: 68
Day 30: Create Azure SQL Database 70
Task Description: 70
Solution: 70
Day 31: Deploying and Managing a Web Application 72
Task Description: 72
Solution: 72
Day 32: Synchronizing Containers Using the CLI 74
Task Description: 74
Solution: 74
Day 33: Integrating Virtual Machines with Application Load Balancer 77
Task Description: 77
Solution: 77
Day 34: Enabling Internet Connectivity for Virtual Machines 81
Task Description: 81
Solution: 81
Day 35: Configuring Virtual Network Peering 84
Task Description: 84
Solution: 84
Day 36: Managing Storage Lifecycle in Azure 87
Task Description: 87
Solution: 87
Day 37: Setting Up MySQL on a Virtual Machine in Azure 90
Task Description: 90
Solution: 91
Day 38: Running Containers on Azure Virtual Machines 94
Task Description: 94
Solution: 94
Day 39: Deploying a Static Website Using Containers on Azure 97
Task Description: 97
Solution: 97
Day 40: Managing Secrets with Azure Key Vault 100
Task Description: 100
Solution: 101
Day 41: Working with Azure Table Storage 103

Task Description: 103
Solution: 103
Day 42: Backup and Delete Azure Storage Blob Container 106
Task Description: 106
Solution: 106
Day 43: Configuring Azure VM with Application Gateway 108
Task Description: 108
Solution: 109
Day 44: Integrating Azure Event Hub with Virtual Machines 114
Task Description: 114
Solution: 114
Day 45: Azure Kubernetes Service (AKS) Setup and Management 117
Task Description: 117
Solution: 117
Day 46: EventHub to Blob Storage Integration Setup 119
Task Description: 119
Solution: 120
Day 47: SQL Database Migration and Setup 124
Task Description: 124
Solution: 125
Day 48: VM and ACR Integration for Storage 128
Task Description: 128
Solution: 128
Day 49: VM Setup with Web Storage Integration 134
Task Description: 134
Solution: 135
Day 50: VM Setup and Configuration for Azure Application Gateway 140
Task Description: 140
Solution: 140

Day 01: Create SSH Key Pair for Azure Virtual Machine

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to the Azure cloud. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition. To achieve this, they have segmented large tasks into smaller, more manageable units. This granular approach enables the team to execute the migration in gradual phases, ensuring smoother implementation and minimizing disruption to ongoing operations. By breaking down the migration into smaller tasks, the Nautilus DevOps team can systematically progress through each stage, allowing for better control, risk mitigation, and optimization of resources throughout the migration process.

For this task, create an SSH key pair with the following requirements:

- The name of the SSH key pair should be `nautilus-kp`.
- The key pair `type` must be `rsa`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Notes: Create the resource according to the **Resource Group**.

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo $RG
```

Create the Azure SSH Key Resource

Generate SSH Key Pair in local

```
ssh-keygen -t rsa -f ~/.ssh/nautilus-kp -N ""
```

Breakdown of the command:

- -t rsa: Specifies the protocol/algorithm type (RSA).
- -f ~/.ssh/nautilus-kp: Sets the filename and location for the key.
- -N "": Sets an empty passphrase (standard for many automated DevOps tasks, though use a passphrase in production for extra security).

Upload to Azure

```
az sshkey create --name "nautilus-kp" --resource-group "$RG" --public-key
"@~/.ssh/nautilus-kp.pub"
```

Note: The `@` symbol is crucial—it tells Azure to read the content of the file.

Verify the Azure Resource

```
az sshkey list --resource-group "$RG" --query
"[?name=='nautilus-kp'].{Name:name, Location:location,
ResourceGroup:resourceGroup}" --output table
```

Verify Local Files

```
ls -l ~/.ssh/nautilus-kp*
```

Day 02: Create an Azure Virtual Machine

Task Description:

The Nautilus DevOps team is planning to migrate a portion of their infrastructure to the Azure cloud incrementally. As part of this migration, you are tasked with creating an Azure Virtual Machine (VM).

The requirements are:

1. Use the existing resource group.
2. The VM name must be `devops-vm`, it should be in `eastus` region.
3. Use the `Ubuntu 22.04 LTS` image for the VM.
4. The VM size must be `Standard_B1s`.
5. Attach a default Network Security Group (NSG) that allows inbound SSH (port 22).
6. Attach a 30 GB storage disk of type `Standard HDD`.
7. The rest of the configurations should remain as default.

After completing these steps, make sure you can SSH into the virtual machine.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure Portal

VM Creation

- Log in to the Azure Portal
- Search for **Virtual Machines** in the top search bar and click **+ Create** > **Azure virtual machine**.
- **Resource Group:** Select our existing group (e.g., `kml_rg_main-...`).
- **Virtual machine name:** `devops-vm`
- **Region:** `(US) East US` (Ensure this is East US, even if your RG is elsewhere).
- **Availability options:** No infrastructure redundancy required.
- **Security type:** Standard.
- **Image:** `Ubuntu Server 22.04 LTS - x64 Gen2`.
- **Size:** Click "See all sizes" and select **Standard_B1s** (1 vCPU, 1 GiB RAM).
- **Authentication type:** Select **SSH public key**.
- **Username:** Set as `azureuser`
- **SSH public key source:** Select **Generate New Key pair**. Named it **devops-vm-key**
- **OS disk type:** Change the dropdown from Premium SSD to **Standard HDD**.
- **Delete with VM:** Checked (default).
- **Key management:** Platform-managed key (default).
- *Note:* The default size for Ubuntu is usually 30GB. If it defaults to something else, click **"Advanced"** to ensure it is set to **30 GB**.
- **Virtual network/Subnet:** Let it default to the values associated with our Resource Group.
- **Public IP:** Let it create a new one (default).
- **NIC network security group:** Select **Basic**.
- **Public inbound ports:** Select **Allow selected ports**.
- **Select inbound ports:** Ensure **SSH (22)** is checked in the dropdown.
    - *This action automatically creates the NSG rule you need.*
- Keep all settings as **Default** as per the task instructions.
- Ensure "Enable boot diagnostics" is on (default) for troubleshooting.
- Click **Review + create**. Azure will run a validation check.
- Once "Validation passed" appears, verify the summary (Standard_B1s, East US, SSH port 22).
- Click **Create**.

Verification

1. Click **Go to resource**.
2. Copy the **Public IP address**.
3. Go back to our **azure-client** terminal and test the connection:
4. Bash

```
ssh -i ~/.ssh/devops-vm-key azureuser@<VM-Public-IP>
```

Day 03: Create VM using Azure CLI

Task Description:

The Nautilus DevOps team is in the process of migrating some of their workloads to Azure. One of the tasks involves creating a new Virtual Machine (VM) using the Azure CLI. The team does not have access to the Azure portal but can manage Azure resources via the `azure-client` host (the landing host for this lab).

1. Create a new Azure Virtual Machine named `xfusion-vm` using the Azure CLI.
2. Use the `Ubuntu22.04` image and set the VM size to `Standard_B2s`.
3. Make sure the admin username is set to `azureuser` and SSH keys are generated for secure access.
4. Use `Standard_LRS` storage account, disk size must be `30GB` and ensure the VM `xfusion-vm` is in the `running` state after creation.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.
If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Command to create VM

```
az vm create --resource-group "$RG" --name "xfusion-vm" --location "westus"
--image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B2s" --admin-username "azureuser" --generate-ssh-keys
--public-ip-sku Standard --nsg-rule "SSH" --os-disk-delete-option "Delete"
--os-disk-name "xfusion-vm-os-disk" --storage-sku "Standard_LRS"
```

Verify

```
az vm show --resource-group "$RG" --name "xfusion-vm" --show-details
--query "{Name:name, Status:powerState, Size:hardwareProfile.vmSize,
Image:storageProfile.imageReference.offer,
OS_Disk_GB:storageProfile.osDisk.diskSizeGb,
Storage_Type:storageProfile.osDisk.managedDisk.storageAccountType}" -o
table
```

Day 04: Create a Virtual Network (VNet) in Azure

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to the Azure cloud. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition. To achieve this, they have segmented large tasks into smaller, more manageable units. This granular approach enables the team to execute the migration in gradual phases, ensuring smoother implementation and minimizing disruption to ongoing operations.

Create a Virtual Network (VNet) named `devops-vnet` in the `southcentralus` region with any `IPv4` CIDR block.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI
Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Virtual Network (VNet)

```
az network vnet create --resource-group "$RG" --name "devops-vnet"
--location "southcentralus" --address-prefixes "10.0.0.0/16" --subnet-name
"default-subnet" --subnet-prefixes "10.0.1.0/24"
```

Verify

```
az network vnet show --resource-group "$RG" --name "devops-vnet" --query
"{Name:name, Location:location, AddressSpace:addressSpace.addressPrefixes[0],
Status:provisioningState}" -o table
```

Day 05: Create a Virtual Network (IPv4) in Azure

Task Description:

The Nautilus DevOps team is strategically planning the migration of a portion of their infrastructure to the Azure cloud. Acknowledging the magnitude of this endeavor, they have chosen to tackle the migration incrementally rather than as a single, massive transition. Their approach involves creating Virtual Networks (VNets) as the initial step, as they will be provisioning various services under different VNets.

Create a Virtual Network (VNet) named `datacenter-vnet` in the `eastus` region with `192.168.0.0/24` IPv4 CIDR.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Virtual Network (VNet)

```
az network vnet create --resource-group "$RG" --name "datacenter-vnet"
--location "eastus" --address-prefixes "192.168.0.0/24" --subnet-name
"default-subnet" --subnet-prefixes "192.168.0.0/28"
```

Verify

```
az network vnet show --resource-group "$RG" --name "datacenter-vnet" --query
"{Name:name, Location:location, AddressSpace:addressSpace.addressPrefixes[0],
Status:provisioningState}" -o table
```

Day 06: Create a Subnet in Azure Virtual Network

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to the Azure cloud. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition.

For this task, create a Virtual Network (VNet) named `nautilus-vnet` and one subnet named `nautilus-subnet` within the VNet in the `centralus` region. Make sure the `IPv4 address range` is `10.0.0.0/16`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Virtual Network (VNet)

```
az network vnet create --resource-group "$RG" --name "nautilus-vnet"
--location "centralus" --address-prefixes "10.0.0.0/16" --subnet-name
"nautilus-subnet" --subnet-prefixes "10.0.1.0/24"
```

Verify

Get Vnet details

```
az network vnet show --resource-group "$RG" --name "nautilus-vnet" --query
"{Name:name, Location:location, AddressSpace:addressSpace.addressPrefixes[0],
Status:provisioningState}" -o table
```

Get Subnet details

```
az network vnet subnet show --resource-group "$RG" --vnet-name
"nautilus-vnet" --name "nautilus-subnet" --output table
```

Day 07: Create a Public IP Address for Azure VM

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to the Azure cloud. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition. By breaking down the migration into smaller tasks, the Nautilus DevOps team can systematically progress through each stage, allowing for better control, risk mitigation, and optimization of resources throughout the migration process.

For this task, allocate a `Public IP` address, name it as `datacenter-pip`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create Public IP

```
az network public-ip create --resource-group "$RG" --name "datacenter-pip"
--location "eastus" --allocation-method "Static" --sku "Standard" --version
IPv4
```

Verify

Get the IP

```
az network public-ip show --resource-group "$RG" --name "datacenter-pip"
--query ipAddress --output tsv
```

Details of Public IP

```
az network public-ip show --resource-group "$RG" --name "datacenter-pip"
--output table
```

Day 08: Attach Managed Disk to Azure Virtual Machine

Task Description:

The Nautilus devops team is migrating services to Azure. They are breaking down tasks to ensure better control and optimization. You are tasked with attaching an existing data disk to a virtual machine (VM).
An existing VM named `devops-vm` and a managed disk named `devops-disk` already exist in the `centralus` region.

- Attach the disk `devops-disk` to the VM `devops-vm` as a data disk.
- Ensure the disk is attached to the VM `devops-vm`.

Make sure that the virtual machine initialization has been completed before submitting this task.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Attach the Existing Disk

```
az vm disk attach --resource-group "$RG" --vm-name "devops-vm" --name
"devops-disk"
```

Verify the Attachment

```
az vm show --resource-group "$RG" --name "devops-vm" --query
"storageProfile.dataDisks[?name=='devops-disk']" --output table
```

Day 09: Attach Network Interface Card (NIC) to Azure Virtual Machine

Task Description:

The nautilus DevOps team is migrating services to Azure. They are breaking down tasks to ensure better control and optimization. You are tasked with attaching an existing network interface (NIC) to a virtual machine (VM).
An existing VM named `nautilus-vm` and a network interface named `nautilus-nic` already exist in the `southcentralus` region.

- Attach the network interface `nautilus-nic` to the VM `nautilus-vm`.
- Ensure the NIC's status is `attached` before submitting the task.

Make sure that the virtual machine initialization has been completed before submitting this task.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Attachment of NIC to the VM

Before adding NIC to VM, We must stop the VM

```
az vm deallocate --resource-group "$RG" --name "nautilus-vm"
```

Attached the NIC to VM

```
az vm nic add --resource-group "$RG" --vm-name "nautilus-vm" --nics
"nautilus-nic"
```

Start the VM

```
az vm start --resource-group "$RG" --name "nautilus-vm"
```

Verification

Check NIC List

```
az vm show --resource-group "$RG" --name "nautilus-vm" --query
"networkProfile.networkInterfaces[].id" -o table
```

Day 10: Attach Public IP to Azure Virtual Machine

Task Description:

The Nautilus DevOps team has already set up a virtual machine and allocated a public IP address. The final task is to attach this public IP to the VM's network interface card (NIC).
An existing VM named `devops-vm-pip` and a public IP address named `devops-pip` already exist.
Attach the public IP `devops-pip` to the network interface of the VM `devops-vm-pip`.
Make sure the VM is properly assigned the public IP.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Get the Network Interface Card name

```
# Capture the full ID first
FULL_ID=$(az vm nic list --resource-group "$RG" --vm-name "devops-vm-pip"
--query "[0].id" -o tsv)

# Extract just the name
NIC_NAME=$(basename $FULL_ID)

echo $NIC_NAME
```

Get the IP configuration name

```
IP_CONFIG=$(az network nic ip-config list --resource-group "$RG" --nic-name
"$NIC_NAME" --query "[0].name" -o tsv)

echo $IP_CONFIG
```

Attach the Public IP

```
az network nic ip-config update --resource-group "$RG" --nic-name
"$NIC_NAME" --name "$IP_CONFIG" --public-ip-address "devops-pip"
```

Verification

```
az vm list-ip-addresses --resource-group "$RG" --name "devops-vm-pip"
--output table
```

Day 11: Change Azure Virtual Machine Size Using Console

Task Description:

The Nautilus Devops team is migrating a portion of their infrastructure to Azure. During the migration, they have created several virtual machines (VMs) in different regions. The team has identified one VM that is underutilized and has decided to change its size to optimize resource usage.

1. Change the VM size from `Standard_B1s` to `Standard_B2s` for the virtual machine named `nautilus-vm`.
2. Ensure the VM is in the `running` state after the size change is complete.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

Using Azure Console

1. Sign in to the Azure Portal
2. In the top search bar, type **Virtual machines** and select it from the services list.
3. Click on the name **nautilus-vm** from the list of virtual machines.
4. On the left-hand sidebar menu, scroll down to the **Availability + scale** section.
5. Click on Size.
6. In the list of available sizes, search for or select Standard_B2s.
7. Select the **Standard_B2s** row and click the **Resize** button at the bottom of the page.
8. Azure will automatically begin the resizing process. This involves a reboot of the virtual machine.
9. Wait for the notification in the top right (bell icon) to say "Successfully resized virtual machine."
10. Go back to the Overview page of **nautilus-vm**.
11. Check the Status. If it says "Stopped" or "Deallocated," click the Start button at the top of the page to ensure it is in the Running state.

Day 12: Add and Manage Tags for Azure Virtual Machines

Task Description:

The Nautilus DevOps team is migrating a portion of their infrastructure to Azure. During the migration, they have created several virtual machines (VMs) in different regions. The team has identified one VM that is not tagged properly so they decided to tag it as needed.
Add the tag `Environment=dev` to the virtual machine named `datacenter-vm`.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Get the Resource ID of the VM

```
VM_ID=$(az vm show --resource-group "$RG" --name "datacenter-vm" --query id
-o tsv)
```

Add the Tag

Use the `az tag update` command with the **Merge** operation. This is safer than `az tag create` because it appends the new tag without deleting any existing tags that might already be on the VM.

```
az tag update --resource-id "$VM_ID" --operation Merge --tags
Environment=dev
```

Verify the Tag

```
az vm show --resource-group "$RG" --name "datacenter-vm" --query "tags" -o
json
```

Day 13: SSH into an Azure Virtual Machine

Task Description:

The Nautilus DevOps team is working on setting up secure SSH access for their virtual machines in Azure. One of the requirements is to add the SSH public key of the root user from the Azure client host (landing host) to the `nautilus-vm` Azure VM's `authorized_keys` file. This ensures secure and password-less SSH access to the VM.

Task Details:

- The VM is named `nautilus-vm` and is running in the `centralus` region. The default SSH user is `azureuser` — use this user to connect to the VM.
- You need to add the root user's SSH public key from the Azure client host to the `authorized_keys` file of the VM's root user.
- The SSH public key of the root user on the Azure client host is located at `/root/.ssh/id_rsa.pub`.
- Copy the public key located at `/root/.ssh/id_rsa.pub` on the Azure client host to the `authorized_keys` file of the root user on `nautilus-vm`.
- Ensure that the proper permissions for the `.ssh` folder and `authorized_keys` file are set on the VM.
- After adding the public key, make sure that you are able to SSH into the `nautilus-vm` VM as the `root` user from the Azure client host without needing a password.

Important Notes:

- You may need to adjust the firewall or security group rules for the VM to allow SSH access.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.

Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Retrieve the VM's Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "nautilus-vm" --query publicIps -o tsv)

echo "Public IP: $VM_IP"
```

Copy the Root Public Key

Store the key in a variable

```
PUB_KEY=$(cat /root/.ssh/id_rsa.pub)
```

Connect as **azureuser** and append the key to the **root** user's **authorized_keys**

```
ssh -o StrictHostKeyChecking=no azureuser@$VM_IP "sudo mkdir -p /root/.ssh
&& echo '$PUB_KEY' | sudo tee -a /root/.ssh/authorized_keys"
```

Set Correct Permissions

```
ssh azureuser@$VM_IP "sudo chmod 700 /root/.ssh && sudo chmod 600
/root/.ssh/authorized_keys && sudo chown -R root:root /root/.ssh"
```

Enable Root Login in SSH Config

SSH into nautilus-vm through azureuser, then edit the sshd config file

```
sudo nano /etc/ssh/sshd_config
```

Find the line `#PermitRootLogin prohibit-password` or `PermitRootLogin no`.
Change it to: **PermitRootLogin yes**
Save the file

Restart the SSH service to apply changes

```
sudo systemctl restart ssh
```

Clean up the **authorized_keys** file
We need to remove that **forced command line** so that only our clean public key remains.

Verify Password-less Access
Run this from the **azure-client** host as the root user

```
ssh root@$VM_IP
```

Day 14: Create and Attach Managed Disks in Azure

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to the Azure cloud. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition.
Create a managed disk with the following requirements:

- Name of the disk should be `nautilus-disk`.
- Disk `type` must be `Standard_LRS`.
- Disk `size` must be `2 GiB`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Managed Disk

```
az disk create --resource-group "$RG" --name "nautilus-disk" --sku
"Standard_LRS" --size-gb 2 --location "eastus"
```

Verify

```
az disk show --resource-group "$RG" --name "nautilus-disk" --query
"{Name:name, Size_GiB:diskSizeGb, Tier:sku.name, State:provisioningState}"
-o table
```

Day 15: Create and Configure Network Security Group (NSG) in Azure

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to the Azure cloud. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition.

For this task, create a network security group (NSG) with the following requirements:

- Name of the NSG should be `xfusion-nsg`.
- Add an inbound security rule named `Allow-HTTP` for `HTTP` service on port `80`, with the source CIDR range of `0.0.0.0/0`.
- Add another inbound security rule named `Allow-SSH` for `SSH` service on port `22`, with the source CIDR range of `0.0.0.0/0`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Network Security Group

```
az network nsg create --resource-group "$RG" --name "xfusion-nsg"
```

Add the Inbound Security Rules

Add HTTP Rule (Port 80)

```
az network nsg rule create --resource-group "$RG" --nsg-name "xfusion-nsg"
--name "Allow-HTTP" --priority 100 --access "Allow" --direction "Inbound"
--source-address-prefixes "0.0.0.0/0" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 80 --protocol
"Tcp"
```

Add SSH Rule (Port 22)

```
az network nsg rule create --resource-group "$RG" --nsg-name "xfusion-nsg"
--name "Allow-SSH" --priority 110 --access "Allow" --direction "Inbound"
--source-address-prefixes "0.0.0.0/0" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 22 --protocol
"Tcp"
```

Verify

```
az network nsg rule list --resource-group "$RG" --nsg-name "xfusion-nsg"
--query "[].{Name:name, Port:destinationPortRange, Access:access,
Priority:priority}" -o table
```

Day 16: Create a Private Azure Blob Storage Container

Task Description:

As part of the data migration process, the Nautilus DevOps team is actively creating several storage containers on Azure. They plan to utilize private Blob containers to store the relevant data. Given the ongoing migration of other infrastructure to Azure, it is logical to consolidate data storage within the Azure environment as well.

Create a new storage account named `datacenterst8750` and a `private` Blob container named `datacenter-blob-18035` within the storage account.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Storage Account

Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "datacenterst8750" --resource-group "$RG"
--location "eastus" --sku "Standard_LRS" --kind "StorageV2"
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='datacenterst8750'].name" -o tsv)
```

Create the Private Blob Container

By default, containers created via the CLI are private (no public read access)

```
az storage container create --name "datacenter-blob-18035" --account-name
"$ST_ACCOUNT"
```

Verify

```
az storage container show --name "datacenter-blob-18035" --account-name
"$ST_ACCOUNT" --query "{Name:name, PublicAccess:properties.publicAccess}"
-o table
```

Day 17: Create a Public Azure Blob Storage Container

Task Description:

As part of the data migration process, the Nautilus DevOps team is actively creating several storage containers on Azure. They plan to utilize public Blob containers to store the relevant data. Given the ongoing migration of other infrastructure to Azure, it is logical to consolidate data storage within the Azure environment as well.

Create a new storage account named `xfusionst9708` and a `public` Blob container named `xfusion-blob-21258` within the storage account. Make sure `anonymous read access for containers and blobs` is enabled.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Storage Account

Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "xfusionst9708" --resource-group "$RG"
--location "eastus" --sku "Standard_LRS" --kind "StorageV2"
--allow-blob-public-access true
```

The `--allow-blob-public-access` flag is the crucial part here. Without it, we cannot make the container public.

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='xfusionst9708'].name" -o tsv)
```

Create the Public Blob Container

```
az storage container create --name "xfusion-blob-21258" --account-name
"$ST_ACCOUNT" --public-access container
```

Store Public Blob Container name in variable

```
PUBLIC_BLOB_CONTAINER=$(az storage container list --account-name
"$ST_ACCOUNT" --query "[?name=='xfusion-blob-21258'].name" -o tsv)

echo "Target Public Container: $PUBLIC_BLOB_CONTAINER"
```

Verify

```
az storage container show --name "$PUBLIC_BLOB_CONTAINER" --account-name
"$ST_ACCOUNT" --query "{Name:name, PublicAccess:properties.publicAccess}"
-o table
```

Day 18: Copy Data to an Azure Blob Storage Container

Task Description:

The Nautilus DevOps team is presently immersed in data migrations, transferring data from on-premise storage systems to Azure Blob containers. They have recently received some data that they intend to copy to one of the Blob containers.

A Blob container named `datacenter-blob-13439` already exists in the `centralus` region under the storage account `datacenterst14220`. Copy the file `/tmp/datacenter.txt` to the Blob container `datacenter-blob-13439`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='datacenterst14220'].name" -o tsv)

echo $ST_ACCOUNT
```

Store Blob Container name in variable

```
BLOB_CONTAINER=$(az storage container list --account-name "$ST_ACCOUNT"
--query "[?name=='datacenter-blob-13439'].name" -o tsv)

echo "Target Blob Container: $BLOB_CONTAINER"
```

Upload the File

```
az storage blob upload --account-name "$ST_ACCOUNT" --container-name
"$BLOB_CONTAINER" --name "datacenter.txt" --file "/tmp/datacenter.txt"
--auth-mode login
```

Verify

```
az storage blob list --account-name "$ST_ACCOUNT" --container-name
"$CONTAINER" --auth-mode login --output table
```

Day 19: Convert Public Azure Blob Container to Private

Task Description:

The Nautilus DevOps team has been using Azure Blob Storage to manage their data. Recently, they realized that one of their containers, currently public, needs to be restricted for internal use only. Your task is to convert a public Azure Blob container to private.

Two blob containers named `devops-container-3911` and `devops-priv-16902` are available in the `eastus` region within the storage account `devopsst15621`. The `devops-container-3911` is currently public, and `devops-priv-16902` is private.

1. Convert the blob container `devops-container-3911` from public to private while leaving `devops-priv-16902` unchanged.
2. Make sure the access level for `devops-container-3911` is set to `private` with no public access.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='devopsst15621'].name" -o tsv)

echo $ST_ACCOUNT
```

Store Public Blob Container name in variable

```
PUBLIC_BLOB_CONTAINER=$(az storage container list --account-name
"$ST_ACCOUNT" --query "[?name=='devops-container-3911'].name" -o tsv)

echo "Target Blob Container: $PUBLIC_BLOB_CONTAINER"
```

Convert the Container to Private

Use the `az storage container set-permission` command. Setting the `--public-access` parameter to `off` removes all anonymous access.

```
az storage container set-permission --account-name "$ST_ACCOUNT" --name
"$PUBLIC_BLOB_CONTAINER" --public-access off --auth-mode login
```

Verify

```
# Check the status of both containers

az storage container list --account-name "$ST_ACCOUNT" --query
"[].{Name:name, PublicAccess:properties.publicAccess}" -o table
```

Day 20: Deploy Azure Resources Using ARM Template

Task Description:

You are tasked with modifying an ARM template for deploying a virtual network. The current template is located in the `/root/arm-templates` directory under the filename `vnet-deployment-template.json`. You need to make the following changes to the template:

1. Change the name and `displayName` tag of the virtual network to `arm-vnet-xfusion`.
2. Update the `addressPrefixes` to `192.168.0.0/16`.
3. Add one more tag named `Environment` with value `KKE-xfusion`.

After making these changes, you need to deploy the ARM template using the Azure CLI.

Use the following command to find out the resource group to use:

```
az group list --query '[].name' --output table | grep 'kml'
```

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Edit the ARM Template

Open the template file

```
sudo vi /root/arm-templates/vnet-deployment-template.json
```

After making the changes, resource block should look similar to this

```
{
    "$schema":
"https://schema.management.azure.com/schemas/2019-04-01/deploymentTemplate.
json#",
    "contentVersion": "1.0.0.0",
    "parameters": {},
    "functions": [],
    "variables": {},
    "resources": [
        {
            "name": "arm-vnet-xfusion",
            "type": "Microsoft.Network/virtualNetworks",
            "apiVersion": "2023-11-01",
            "location": "[resourceGroup().location]",
            "tags": {
                "displayName": "arm-vnet-xfusion",
                "Environment": "KKE-xfusion"
             },
            "properties": {
                "addressSpace": {
                    "addressPrefixes": [
                        "192.168.0.0/16"
                    ]
                }
            }
        }
    ],
    "outputs": {
    }
}
```

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Deploy the Template

```
az deployment group create --resource-group "$RG" --template-file
/root/arm-templates/vnet-deployment-template.json
```

Verifications

```
az network vnet show --resource-group "$RG" --name arm-vnet-xfusion --query
"{AddressSpace:addressSpace.addressPrefixes, Tags:tags}" -o json
```

Day 21: Assigning Public IP to Virtual Machines

Task Description:

The Nautilus DevOps Team has received a new request from the Development Team to set up a new Azure Virtual Machine (VM). This VM will be used to host a new application that requires a stable public IP address. To ensure that the VM has a consistent public IP, a Static Public IP address needs to be associated with it. The VM will be named `xfusion-vm`, and the Static Public IP will be named `xfusion-pip`. This setup will help the Development Team to have a reliable and consistent access point for their application.

1. Create an Azure VM named `xfusion-vm` using any available Ubuntu image, with the VM size `Standard_B1s`.
2. Generate an SSH public key on the `azure-client` host and associate it with the VM for SSH access.
3. Associate a Static Public IP address named `xfusion-pip` with this VM.
4. Ensure the VM is accessible via SSH using the generated public key.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Generate the SSH Key

```
ssh-keygen -t rsa -b 4096 -N "" -f /root/.ssh/id_rsa
```

Create the Static Public IP

```
STATIC_IP=$(az network public-ip create --resource-group "$RG" --location
"centralus" --name "xfusion-pip" --allocation-method Static --sku Standard
--query publicIp.ipAddress -o tsv)

echo $STATIC_IP
```

Create the Virtual Machine

```
az vm create --resource-group "$RG" --name "xfusion-vm" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B1s" --location "centralus" --admin-username "azureuser"
--ssh-key-values /root/.ssh/id_rsa.pub --public-ip-address "xfusion-pip"
--nsg-rule "SSH" --storage-sku "Standard_LRS"
```

Verify

Test SSH access

```
ssh azureuser@$STATIC_IP
```

Day 22: Configuring Instances with User Data

Task Description:

The Nautilus DevOps Team is working on setting up a new virtual machine (VM) to host a web server for a critical application. The team lead has requested you to create an Azure VM that will serve as a web server using Nginx. This VM will be part of the initial infrastructure setup for the Nautilus project. Ensuring that the server is correctly configured and accessible from the internet is crucial for the upcoming deployment phase.

As a member of the Nautilus DevOps Team, your task is to create a VM with the following specifications:

Instance Name: The VM must be named `nautilus-vm`.
Image: Use any available Ubuntu image to create this VM.
Custom Script Extension/User Data: Configure the VM to run a custom script during its launch. This script should:

- Install the Nginx package.
- Start the Nginx service.

Network Security Group (NSG): Ensure that the VM allows HTTP traffic on port `80` from the internet.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure Console

1. Log into the Azure Portal.
2. Search for **Virtual Machines** and click **Create** > **Azure virtual machine**.
3. **Project Details:** Select your subscription and the **Resource Group** (look for the one containing `kml`).
4. **Instance Details:**
   - **VM Name:** `nautilus-vm`
   - **Region:** `East US`
   - **Image:** Select `Ubuntu Server 22.04 LTS - x64 Gen2`.
   - **Size:** `Standard_B1s`.
5. **Authentication type:** Select `SSH public key`.
6. **Username:** `azureuser`.
7. **SSH public key source:** Select `Use existing public key`.
8. **Stored key:** Paste the contents of your `/root/.ssh/id_rsa.pub` from the `azure-client` host.
9. Click the **Networking** tab at the top.
10. **NIC network security group:** Ensure `Basic` is selected.
11. **Public inbound ports:** Select `Allow selected ports`.
12. **Select inbound ports:** Check both **SSH (22)** and **HTTP (80)**.
13. Click the **Advanced** tab.
14. Scroll down to the **User data** section.
15. Check the box **Enable user data**.
16. Paste your script into the text box:

```
#!/bin/bash
apt-get update
apt-get install -y nginx
systemctl start nginx
systemctl enable nginx
```

Day 23: Automating User Data Configuration Using the CLI

Task Description:

The Nautilus DevOps Team is working on setting up a new virtual machine (VM) to host a web server for a critical application. The team lead has requested you to create an Azure VM that will serve as a web server using Nginx. This VM will be part of the initial infrastructure setup for the Nautilus project. Ensuring that the server is correctly configured and accessible from the internet is crucial for the upcoming deployment phase.

Your task is to create a VM using Azure CLI with the following specifications:

Instance Name: The VM must be named `devops-vm`.
Image: Use any available Ubuntu image to create this VM.
Custom Script Extension/User Data: Configure the VM to run a custom script during its launch. This script should:

- Install the Nginx package.
- Start the Nginx service.

Network Security Group (NSG): Ensure that the VM allows HTTP traffic on port `80` from the internet.

Instructions: Use Azure CLI commands to set up the VM in the specified configuration.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Prepare the User Data Script

Create script file

```
touch userscript.txt
```

Add the following content in the file

```
#!/bin/bash

sudo apt-get update
sudo apt-get install -y nginx

sudo systemctl start nginx
sudo systemctl enable nginx
```

Create the VM

```
az vm create --resource-group "$RG" --name "devops-vm" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--location "eastus" --admin-username "azureuser" --size "Standard_B1s"
--generate-ssh-keys --public-ip-sku Standard --custom-data userscript.txt
--storage-sku "Standard_LRS"
```

Configure NSG Rule

Identify NSG Name

```
NSG_NAME=$(az network nsg list --resource-group "$RG" --query
"[?contains(name, 'devops-vm')].name" -o tsv)
```

```
echo "Found NSG: $NSG_NAME"
```

Create the Allow-HTTP Rule

```
az network nsg rule create --resource-group "$RG" --nsg-name "$NSG_NAME"
--name "Allow-HTTP" --protocol "Tcp" --direction "Inbound" --priority 1001
--source-address-prefixes "*" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 80 --access
"Allow"
```

Verify

Retrieve the Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "devops-vm" --query "publicIps" -o tsv)
```

Test the HTTP response

```
curl -I http://$VM_IP
```

Day 24: Securing Virtual Machine SSH Access

Task Description:

The Nautilus DevOps team needs to set up a new Virtual Machine (VM) on the Azure cloud that can be accessed securely from their landing host (`azure-client`). Follow the steps below to complete this task:

1. **Create an SSH Key**: On the `azure-client` host, check if an SSH key already exists. If it doesn't exist, create a new SSH key on the `azure-client` host that will be used for password-less SSH access.
2. **Create a Virtual Machine**: Use the Azure Portal or Azure CLI to create a new Virtual Machine named `nautilus-vm` in the `westus` region. Set the VM size to **Standard_B1s** and configure the VM with **SSH access** for the `azureuser` account using the newly created SSH key.
3. **Configure SSH Access**: Ensure that the SSH key from the `azure-client` host is added to the `azureuser` account on `nautilus-vm`, enabling secure, password-less SSH access from the `azure-client` host.
4. **Verify Connectivity**: Test the connection from `azure-client` to `nautilus-vm` using SSH to confirm that password-less access has been set up correctly.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Check if key exists

```
ls -l ~/.ssh/id_rsa.pub
```

If it doesn't exist, create it

```
ssh-keygen -t rsa -b 4096 -N "" -f ~/.ssh/id_rsa
```

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the VM

```
az vm create --resource-group "$RG" --name "nautilus-vm" --location
"westus" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B1s" --admin-username "azureuser" --ssh-key-values
~/.ssh/id_rsa.pub --storage-sku "Standard_LRS" --public-ip-sku Standard
```

Get the Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "nautilus-vm" --query "publicIps" -o tsv)
```

Connect via SSH

```
ssh azureuser@$VM_IP
```

Day 25: Expanding and Managing Disk Storage

Task Description:

The Nautilus DevOps team needs to expand the storage capacity of an existing virtual machine and add an additional data disk to support increased workloads. This task requires resizing the existing VM disk and mounting a new data disk to the VM.

As a member of the team, perform the following steps:

- Expand the existing VM `xfusion-vm` disk from `32Gi` to `64Gi`.
- Also create a new standard HDD data disk named `xfusion-disk` of `64Gi` and mount the disk to VM `xfusion-vm` at location `/mnt/xfusion-disk`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Expand the Existing OS Disk
Azure requires the VM to be **Deallocated** before resizing the OS disk.

Stop and Deallocate the VM

```
az vm deallocate --resource-group "$RG" --name "xfusion-vm"
```

Get the OS Disk name

```
DISK_NAME=$(az vm show -g "$RG" -n "xfusion-vm" --query
"storageProfile.osDisk.name" -o tsv)

echo $DISK_NAME
```

Update the OS Disk size to 64GB

```
az disk update --resource-group "$RG" --name "$DISK_NAME" --size-gb 64
```

Start the VM

```
az vm start --resource-group "$RG" --name "xfusion-vm"
```

Create and Attach the New Data Disk

```
az vm disk attach --resource-group "$RG" --vm-name "xfusion-vm" --name
"xfusion-disk" --new --size-gb 64 --sku "Standard_LRS"
```

--sku "Standard_LRS" defines it as a Standard HDD

Configure the OS Mounting

Get VM IP

```
VM_IP=$(az vm show -d -g "$RG" -n "xfusion-vm" --query "publicIps" -o tsv)

# SSH into VM
ssh azureuser@$VM_IP
```

Inside the VM, run the below commands

First we need to check the OS Disk size, after SSH into VM run

```
df -h
```

This command show the total size of the root directory

Now check the attached disks in the VM

```
lsblk
```

Create the Mount Point

```
sudo mkdir -p /mnt/xfusion-disk
```

Format the Disk
Since the disk is new, we need to put a filesystem on it. We'll use `ext4`

```
sudo mkfs -t ext4 /dev/sdc
```

Mount the Disk

```
sudo mount /dev/sdc /mnt/xfusion-disk
```

Make it Persistent
Disk will disappear from `/mnt/xfusion-disk` when the VM reboots. We need to add it to the `/etc/fstab` file.

First, get the **UUID** of the new disk

```
sudo blkid /dev/sdc
```

Copy the UUID string for **/dev/sdc**
In my case the UUID is: **3c4c2b6c-9a15-46bf-9546-ed3ba21bb4c0**

Then, open the fstab file:

```
sudo nano /etc/fstab
```

Add this line to the bottom of the file

```
UUID=3c4c2b6c-9a15-46bf-9546-ed3ba21bb4c0   /mnt/xfusion-disk   ext4
defaults   0   2
```

Day 26: Deploying Virtual Machines in a Public Virtual Network

Task Description:

The Nautilus DevOps Team has received a request from the Networking Team to set up a new public VNet to support a set of public-facing services. This VNet will host various resources that need to be accessible over the internet. As part of this setup, you need to ensure the VNet has public subnets with automatic public IP assignment for resources. Additionally, a new VM will be launched within this VNet to host public applications that require SSH access. This setup will enable the Networking Team to deploy and manage public-facing applications.

Create a public VNet named `nautilus-pub-vnet`, and a subnet named `nautilus-pub-subnet` under the same, make sure public IP is being auto-assigned to resources under this subnet.

Create a VM named `nautilus-pub-vm` under this VNet. Make sure SSH port `22` is open for this instance and accessible over the internet. ensure that SSH access is configured correctly.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Virtual Network and Subnet

Create the VNet and Subnet

```
az network vnet create --resource-group "$RG" --name "nautilus-pub-vnet"
--address-prefix "10.0.0.0/16" --subnet-name "nautilus-pub-subnet"
--subnet-prefix "10.0.1.0/24"
```

Create the VM with SSH Access

```
az vm create --resource-group "$RG" --name "nautilus-pub-vm" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--location "eastus" --vnet-name "nautilus-pub-vnet" --subnet
"nautilus-pub-subnet" --admin-username "azureuser" --generate-ssh-keys
--size "Standard_B1s" --storage-sku "Standard_LRS" --public-ip-address ""
--public-ip-sku Standard --nsg-rule SSH
```

Create the Public IP resource

```
az network public-ip create --resource-group "$RG" --name
"nautilus-pub-vm-pip" --sku Standard --location "eastus"
```

Get the NIC name associated with VM

```
NIC_NAME=$(az vm show -g "$RG" -n "nautilus-pub-vm" --query
"networkProfile.networkInterfaces[0].id" -o tsv | rev | cut -d'/' -f1 |
rev)
```

Find the Correct IP Config Name

```
IP_CONFIG_NAME=$(az network nic ip-config list --resource-group "$RG"
--nic-name "$NIC_NAME" --query "[0].name" -o tsv)
```

Update the NIC to use the new Public IP

```
az network nic ip-config update --resource-group "$RG" --nic-name
"$NIC_NAME" --name "$IP_CONFIG_NAME" --public-ip-address
"nautilus-pub-vm-pip"
```

Verify

```
VM_IP=$(az vm show -d -g "$RG" -n "nautilus-pub-vm" --query "publicIps" -o
tsv)

echo "The Public IP is: $VM_IP"
```

Test SSH Connectivity

```
ssh azureuser@$VM_IP
```

Day 27: Deploying Virtual Machines in a Private Virtual Network

Task Description:

The Nautilus DevOps team is expanding their Azure infrastructure and requires the setup of a **private Virtual Network** (VNet) along with a subnet. This VNet and subnet configuration will ensure that resources deployed within them remain isolated from external networks and can only communicate within the VNet. Additionally, the team needs to provision a Virtual Machine (VM) under the newly created private VNet. This VM should be accessible over SSH from within the VNet only, allowing for secure communication and resource management within the Azure environment.

- The name of the VNet must be `xfusion-priv-vnet`, create a subnet named `xfusion-priv-subnet` under the same.
- Create a Virtual Machine named `xfusion-priv-vm` under this VNet.
- Create a Network Security Group (NSG) named `xfusion-priv-nsg`, and ensure that the NSG rules for the VM allow access only from within the VNet's CIDR block.

Ensure all resources are created in the `Central US` region.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.

Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Private VNet and Subnet

```
az network vnet create --resource-group "$RG" --name "xfusion-priv-vnet"
--location "centralus" --address-prefix "10.0.0.0/16" --subnet-name
"xfusion-priv-subnet" --subnet-prefix "10.0.1.0/24"
```

Create the Restricted NSG
We need to create the NSG and a rule that explicitly allows SSH only from the VNet's own CIDR block (`10.0.0.0/16`)

Create the NSG

```
az network nsg create --resource-group "$RG" --name "xfusion-priv-nsg"
--location "centralus"
```

Create the inbound rule for Internal SSH only

```
az network nsg rule create --resource-group "$RG" --nsg-name
"xfusion-priv-nsg" --name "Allow-SSH-From-VNet" --priority 1010
--source-port-ranges 22 --source-address-prefixes "10.0.0.0/16"
--destination-port-ranges 22 --destination-address-prefixes "10.0.0.0/16"
--direction Inbound --access Allow --protocol Tcp
```

Provision the Private VM
When creating the VM, we will associate the NSG we just built and explicitly set `--public-ip-address ""` to ensure it has no external footprint.

```
az vm create --resource-group "$RG" --name "xfusion-priv-vm" --location
```

```
"centralus" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--vnet-name "xfusion-priv-vnet" --subnet "xfusion-priv-subnet" --nsg
"xfusion-priv-nsg" --admin-username "azureuser" --generate-ssh-keys
--public-ip-address "" --size "Standard_B1s" --storage-sku "Standard_LRS"
```

Verify

Check for Public IP (Should be empty)

```
az vm show -d -g "$RG" -n "xfusion-priv-vm" --query "publicIps" -o tsv
```

Day 28: Troubleshooting Public Virtual Network Configurations

Task Description:

The Nautilus DevOps Team deployed an Nginx server on an Azure VM in a public VNet named `devops-vnet`. However, the server is still inaccessible from the internet.

Complete the following tasks:

1. **Verify VNet Configuration**: Ensure `devops-vnet` allows internet access.
2. **Attach Public IP**: A public IP named `devops-pip` already exists. Attach this public IP to the VM `devops-vm` to make it accessible from the internet.
3. **Ensure Accessibility**: Confirm the VM `devops-vm` is accessible on port 80.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Attach the Public IP to the VM

Find the NIC name associated with **devops-vm**

```
NIC_NAME=$(az vm show -g "$RG" -n "devops-vm" --query
"networkProfile.networkInterfaces[0].id" -o tsv | rev | cut -d'/' -f1 |
rev)

echo $NIC_NAME
```

Find the specific IP configuration name

```
IP_CONFIG=$(az network nic ip-config list -g "$RG" --nic-name "$NIC_NAME"
--query "[0].name" -o tsv)

echo $IP_CONFIG
```

Attach the existing Public IP

```
az network nic ip-config update --resource-group "$RG" --nic-name
"$NIC_NAME" --name "$IP_CONFIG" --public-ip-address "devops-pip"
```

Configure NSG

```
az vm open-port --resource-group "$RG" --name "devops-vm" --port 80
--priority 1010
```

VM Configuration

Get the Public IP of VM

```
PUBLIC_IP=$(az network public-ip show -g "$RG" -n "devops-pip" --query
ipAddress -o tsv)
```

SSH into your VM

```
ssh azureuser@$PUBLIC_IP
```

Verify Nginx Installation

```
nginx -v
```

If nginx is not installed then installed it

```
sudo apt install nginx
```

Check Status

```
sudo systemctl status nginx
```

Enable Service

```
sudo systemctl enable nginx
```

Check if any firewall rule set to block HTTP port traffic then unblock it
Exit from the VM

Route Table Configuration
devops-rtb is attached to the subnet that block the internet traffic.

Update the Route

```
az network route-table route update --resource-group "$RG"
--route-table-name "devops-rtb" --name "Block-Internet" --next-hop-type
"Internet"
```

Verify
Test the connection

```
curl -I http://$PUBLIC_IP
```

Day 29: Working with Azure Container Registry (ACR)

Task Description:

The Nautilus DevOps team has been tasked with setting up a containerized application. They need to create a Azure Container Registry (ACR) to store their Docker images. Once the repository is created, they will build a Docker image from a Dockerfile located on the `azure-client` host and push this image to the ACR repository. This process is essential for maintaining and deploying containerized applications in a streamlined manner.

1. Create a ACR repository named `nautilusacr23228` under `East US`.
2. Pricing plan must be `Basic`.
3. Dockerfile already exists under `/root/pyapp` directory on `azure-client` host.
4. Build a Docker image using this Dockerfile and push the same to the newly created ACR repo. The image tag must be `latest` i.e `nautilusacr23228:latest`.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Azure Container Registry

```
az acr create --resource-group "$RG" --name "nautilusacr23228" --sku Basic
--location eastus
```

Build and Push the Image

Navigate to the app directory

```
cd /root/pyapp
```

Log in to ACR

```
az acr login --name "nautilusacr23228"
```

Build the image locally

```
docker build -t nautilusacr23228.azurecr.io/nautilusacr23228:latest .
```

Push the image

```
docker push nautilusacr23228.azurecr.io/nautilusacr23228:latest
```

Verify

```
az acr repository list --name nautilusacr23228 --output table
```

Day 30: Create Azure SQL Database

Task Description:

The Nautilus Devops team is strategizing the migration of a portion of their infrastructure to Azure. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition. Recently, they started working on creating and configuring some database instances on Azure.

For this task, create one `publicly` accessible Azure SQL Database instance along with the following details:

1. The name of the Azure SQL Database must be `datacenter-sqldb`.
2. The server name must be `datacenter-server-1691` under `centralus`.
3. The compute + storage configuration should be **Basic (For less demanding workloads)**.
4. The backup storage redundancy should be **Locally-redundant backup storage**.
5. Set the login admin username to `datacenter-admin` and set an appropriate password.
6. Set the database size to **2 GiB**.
7. Keep the rest of the configurations as `default`. Finally, make sure the database is in the `Ready` state before submitting this task.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the SQL Server

First, we create the logical server that will host the database. Make sure to replace `<YourPassword>` with a strong password.

Create the SQL Server in Central US

```
az sql server create --name "datacenter-server-1691" --resource-group "$RG"
--location "centralus" --admin-user "datacenter-admin" --admin-password
"<YourPassword>"
```

Configure Public Access (Firewall)

```
az sql server firewall-rule create --resource-group "$RG" --server
"datacenter-server-1691" --name "AllowAzureServices" --start-ip-address
"0.0.0.0" --end-ip-address "0.0.0.0"
```

Create the SQL Database

```
az sql db create --resource-group "$RG" --server "datacenter-server-1691"
--name "datacenter-sqldb" --edition "Basic" --capacity 5 --max-size 2GB
--backup-storage-redundancy "Local"
```

Verify

```
az sql db show --resource-group "$RG" --server "datacenter-server-1691"
--name "datacenter-sqldb" --query "status" -o tsv
```

Day 31: Deploying and Managing a Web Application

Task Description:

The Nautilus DevOps team is tasked with deploying a Python-based web application on Azure. You need to create a web app using the following specifications:

1) The **Web App** name should be `xfusion-webapp`.
2) It should be created in the **West US** region under the **default resource group**.
3) The **publish** option should be set to `Code`.
4) The **Runtime Stack** should be `Python` with `Linux` as the operating system.
5) Create a new **App Service Plan** named `xfusion-learn-python` with the SKU `Basic B1`.
6) **Application Insights** should be disabled.
7) Add **tags**:

- Name: WebAppLearning
- Environment: Dev

Make sure the web app is in `Running` state after creation.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the App Service Plan
Create the underlying infrastructure plan named **xfusion-learn-python** in the **West US** region with the **B1** pricing tier.

```
az appservice plan create --name "xfusion-learn-python" --resource-group
"$RG" --location "westus" --is-linux --sku B1
```

Create the Python Web App
Create the Web App instance (App Service). We will specify the Python runtime, disable Application Insights, and apply the required tags.

```
az webapp create --name "xfusion-webapp" --resource-group "$RG" --plan
"xfusion-learn-python" --runtime "PYTHON:3.9" --tags Name=WebAppLearning
Environment=Dev
```

Verify

```
az webapp show --name "xfusion-webapp" --resource-group "$RG" --query
"state" --output tsv
```

Day 32: Synchronizing Containers Using the CLI

Task Description:

As part of a data migration project, the team lead has tasked the team with migrating data from an existing Azure Blob container to a new Blob container. The existing container contains a substantial amount of data that must be accurately transferred to the new container. The team is responsible for creating the new Blob container and ensuring that all data from the existing container is copied or synced to the new container completely and accurately. It is imperative to perform thorough verification steps to confirm that all data has been successfully transferred to the new container without any loss or corruption.

As a member of the Nautilus DevOps Team, your task is to perform the following:

Create a New Private Azure Blob Container: Name the container `datacenter-dest-5441` under the storage account `datacenterst21001`.

Data Migration: Migrate the file `datacenter.txt` from the existing `datacenter-source-6093` container to the new `datacenter-dest-5441` container.

Ensure Data Consistency: Ensure that both containers have the file `datacenter.txt` and confirm the file content is identical in both containers.

Use Azure CLI: Use the Azure CLI to perform the creation and data migration tasks.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Create the New Private Blob Container
By default, Azure CLI creates containers with "Off" (Private) public access.

```
az storage container create --name "datacenter-dest-5441" --account-name
"datacenterst21001" --auth-mode login
```

Migrate the Data

```
az storage blob copy start --account-name "datacenterst21001"
--destination-blob "datacenter.txt" --destination-container
"datacenter-dest-5441" --source-container "datacenter-source-6093"
--source-blob "datacenter.txt" --auth-mode login
```

Verify
To ensure the transfer was successful and the data is identical, you should check both the file's existence and its **Content-MD5** hash. Azure calculates this hash during the transfer to ensure integrity.

Check File Existence and Copy Status
Run this to confirm the file exists in the destination and the copy status is success:

```
az storage blob show --account-name "datacenterst21001" --container-name
"datacenter-dest-5441" --name "datacenter.txt" --query "{Name:name,
Status:properties.copy.status, Size:properties.contentLength}" --output
table
```

Compare Content Integrity (MD5 Hashes)
Compare the MD5 hashes of the source and destination files. If the strings match, the content is identical.

Get Hash from Source

```
az storage blob show -c "datacenter-source-6093" -n "datacenter.txt"
--account-name "datacenterst21001" --query
```

```
"properties.contentSettings.contentMd5" -o tsv
```

Get Hash from Destination

```
az storage blob show -c "datacenter-dest-5441" -n "datacenter.txt"
--account-name "datacenterst21001" --query
"properties.contentSettings.contentMd5" -o tsv
```

Day 33: Integrating Virtual Machines with Application Load Balancer

Task Description:

The Nautilus DevOps team is currently working on setting up a simple application on the Azure cloud. They aim to establish an Azure Load Balancer in front of a Virtual Machine (VM) where an Nginx server is currently running. While the Nginx server currently serves a sample page, the team plans to deploy the actual application later.

1. Set up an Azure Load Balancer named `devops-lb`.
2. Configure the Load Balancer's frontend IP configuration with the name `devops-lb-ip` and assign a public IP address with the same name (`devops-lb-ip`).
3. Create a backend pool named `devops-backend-pool` and add the VM running Nginx to this pool.
4. Create a health probe named `devops-health-probe` on port `80` to check the VM's health.
5. Set up a load balancer rule named `devops-lb-rule` to route traffic on port `80` to the backend pool on port `80`.
6. Add an inbound rule to the existing NSG of the VM to allow HTTP traffic on port 80.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Public IP

```
az network public-ip create --resource-group "$RG" --location "eastus"
--name "devops-lb-ip" --sku Standard
```

Create the Load Balancer

```
az network lb create --resource-group "$RG" --location "eastus" --name
"devops-lb" --sku Standard --public-ip-address "devops-lb-ip"
--frontend-ip-name "devops-lb-ip" --backend-pool-name "devops-backend-pool"
```

Configure Health Probe and Rules

Set up the health check to monitor port 80 and the load balancing rule to direct traffic.

Create the Health Probe

```
az network lb probe create --resource-group "$RG" --lb-name "devops-lb"
--name "devops-health-probe" --protocol Tcp --port 80
```

Create the Load Balancer Rule

```
az network lb rule create --resource-group "$RG" --lb-name "devops-lb"
--name "devops-lb-rule" --protocol Tcp --frontend-port 80 --backend-port 80
--frontend-ip-name "devops-lb-ip" --backend-pool-name "devops-backend-pool"
--probe-name "devops-health-probe"
```

Add VM to Backend Pool

To add our existing VM to the pool, you associate its Network Interface (NIC) with the load balancer's backend pool.

List the VM's in Resource Group

```
az vm list --resource-group "$RG" --query "[].name" -o tsv
```

Filter for a Specific VM

```
VM_NAME=$(az vm show --resource-group "$RG" --name "<VM_NAME>" --query
"name" -o tsv)

echo $VM_NAME
```

Get the Network Interface Card name

```
# Capture the full ID first
FULL_ID=$(az vm nic list --resource-group "$RG" --vm-name "$VM_NAME"
--query "[0].id" -o tsv)

# Extract just the name
NIC_NAME=$(basename $FULL_ID)

echo $NIC_NAME
```

Get the IP configuration name

```
IP_CONFIG=$(az network nic ip-config list --resource-group "$RG" --nic-name
"$NIC_NAME" --query "[0].name" -o tsv)

echo $IP_CONFIG
```

Add the VM's NIC to the backend pool

```
az network nic ip-config address-pool add --resource-group "$RG" --lb-name
"devops-lb" --address-pool "devops-backend-pool" --nic-name "$NIC_NAME"
--ip-config-name "$IP_CONFIG"
```

Update NSG for HTTP Traffic

Finally, open port 80 on the existing Network Security Group (NSG) to allow the Load Balancer and external traffic to reach the Nginx server.

Get the NSG name associated with VM

```
NSG_NAME=$(az network nsg list --resource-group "$RG" --query "[0].name" -o
tsv)
```

Check for Existing Port 80 Rules

```
NSG_HTTP_RULE=$(az network nsg rule list --resource-group "$RG" --nsg-name
```

```
"$NSG_NAME" --query "[?destinationPortRange=='80' &&
direction=='Inbound'].name" -o tsv)
```

If HTTP port not exist then added
Add inbound rule for port 80

```
az network nsg rule create --resource-group "$RG" --nsg-name "$NSG_NAME"
--name "AllowHTTPInbound" --priority 100 --protocol Tcp
--destination-port-ranges 80 --access Allow --direction Inbound
```

Day 34: Enabling Internet Connectivity for Virtual Machines

Task Description:

The Nautilus DevOps team has encountered an issue with an Azure VM named `xfusion-vm`. They are unable to install any packages on this VM due to connectivity issues. The team needs to identify the root cause of the problem and resolve it to restore normal operations.

1. Investigate the connectivity issue preventing package installation on the Azure VM `xfusion-vm`.
2. Implement a solution to resolve the connectivity issue and restore package installation capabilities on the VM.

Note: The SSH key required to access the Azure VM is already created and added to the VM's authorized keys. You can find the SSH key at `/root/.ssh/id_rsa` on the `azure-client` host.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Identify the Root Cause
Connectivity issues for package managers are almost always caused by one of three things:

1. **NSG Outbound Rules:** A "Deny All" rule is blocking traffic on ports 80 (HTTP) and 443 (HTTPS).
2. **Route Table (UDR):** A custom route is forcing traffic to a "Blackhole" or an NVA (Firewall) that isn't configured for egress.
3. **Missing Public IP/NAT:** In newer Azure regions, "Default Outbound Access" is disabled, meaning VMs without a Public IP or NAT Gateway cannot reach the internet.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Access the VM
Get the Public IP of VM

```
VM_PUBLIC_IP=$(az vm show -d -g "$RG" -n "xfusion-vm" --query publicIps -o
tsv)
```

SSH into your VM

```
ssh azureuser@$VM_PUBLIC_IP
```

Inside the VM,
test outbound connectivity

```
curl -I https://www.google.com
```

If this hangs or says "Connection timed out", outbound traffic is blocked.

Check and Fix NSG Outbound Rules
The most common culprit is a high-priority "Deny" rule in the Network Security Group (NSG).

Get NSG Name

```
NSG_NAME=$(az network nsg list -g "$RG" --query "[0].name" -o tsv)
```

Get the List of rules

```
az network nsg rule list -g "$RG" --nsg-name "$NSG_NAME" --query
```

```
"[?direction=='Outbound'].{Name:name, Priority:priority, Action:action,
Port:destinationPortRange}" --output table
```

Block-All-Outbound rule added to block all the outbound traffic.

Allow Outbound Traffic

```
az network nsg rule create --resource-group "$RG" --nsg-name "xfusion-nsg"
--name "AllowAppUpdates" --priority 150 --direction Outbound --access Allow
--protocol Tcp --destination-port-ranges 80 443 --description "Allow
outbound traffic for package installations"
```

Verify the Rule is Active

```
az network nsg rule list -g "$RG" --nsg-name "xfusion-nsg" --query
"[?direction=='Outbound'].{Name:name, Priority:priority, Action:access}"
--output table
```

In my case, the issue is with the NSG rule that blocks all the outbound traffic.

Day 35: Configuring Virtual Network Peering

Task Description:

The Nautilus DevOps team has been tasked with demonstrating the use of VNet Peering to enable communication between two VNets. One VNet will be a private VNet that contains a private Azure VM, while the other will be a public VNet containing a publicly accessible Azure VM.

1) **Existing Azure Resources**:
   - Public VM: `devops-pub-vm` is already in the public VNet.
   - Private VNet and VM: `devops-priv-vnet` and `devops-priv-vm` exist in the private VNet with its subnet: `devops-priv-subnet`.

2) **Create VNet Peering**:
   - Create a VNet Peering between the Public VNet and Private VNet.
   - VNet Peering Name: `devops-pub-to-priv-peering`.

3) **Test the Connection**:
   - SSH into the public VM and verify that you can ping the private VM.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Get the Name of the Vnet in the Resource Group

```
az network vnet list -g "$RG" --query "[].name" -o tsv
```

Identify VNet Resource IDs
Get Public VNet ID

```
PUB_VNET_ID=$(az network vnet show -g "$RG" -n "devops-pub-vnet" --query id
-o tsv)

echo $PUB_VNET_ID
```

Get Private VNet ID

```
PRIV_VNET_ID=$(az network vnet show -g "$RG" -n "devops-priv-vnet" --query
id -o tsv)

echo $PRIV_VNET_ID
```

Create VNet Peering
We will create the peering from the Public VNet to the Private VNet. To ensure the connection is fully established, we typically create the reverse link as well.

Create peering from Public to Private

```
az network vnet peering create --name "devops-pub-to-priv-peering"
--resource-group "$RG" --vnet-name "devops-pub-vnet" --remote-vnet
"$PRIV_VNET_ID" --allow-vnet-access
```

The peering in an **"Initiated"** state with the warning **"Remote sync required."** This is normal because VNet peering is not a single-link connection; it requires two separate peering links to be fully functional.
Currently, you have created the "outbound" path from `devops-pub-vnet` to `devops-priv-vnet`, but the "return" path does not exist yet.

Create the peering from the **Private VNet** back to the **Public VNet**. Once both links exist, the status will automatically change to **Connected**.

Create peering from Private to Public (Required for bi-directional traffic)

```
az network vnet peering create --name "devops-priv-to-pub-peering"
--resource-group "$RG" --vnet-name "devops-priv-vnet" --remote-vnet
"$PUB_VNET_ID" --allow-vnet-access
```

Verify Peering Status
Ensure the peering state shows as **Connected**.

```
az network vnet peering show --name "devops-pub-to-priv-peering"
--resource-group "$RG" --vnet-name "devops-pub-vnet" --query "peeringState"
-o tsv
```

Test the Connection
Log into the public VM and attempt to reach the private VM's internal IP.

Get IPs

```
PUB_IP=$(az vm show -d -g "$RG" -n "devops-pub-vm" --query publicIps -o
tsv)

PRIV_IP=$(az vm show -d -g "$RG" -n "devops-priv-vm" --query privateIps -o
tsv)
```

SSH and Ping

```
# SSH into the public VM
ssh azureuser@$PUB_IP

# From inside the public VM, ping the private VM
ping -c 4 <PRIVATE_VM_PRIVATE_IP>
```

Day 36: Managing Storage Lifecycle in Azure

Task Description:

The Nautilus DevOps team needs to optimize data retention costs by automating the deletion of old blobs. They plan to implement Blob Lifecycle Management for a specific container in Azure Storage.
Task:
1) Create a Storage Account:

- Name the storage account `devopsstor10509`.
- Set the **region** to **East US**.
- Use **Locally-redundant storage (LRS)** as the redundancy option.

2) Create a Blob Container:

- Name the container `devops-container10509`.

3) Upload a File to the Container:

- Upload the file named `tempfile.txt` to the container. The file is present under `/root` of the client host.

4) Configure Blob Lifecycle Management:

- Apply a Lifecycle Management rule named `devops-del-rule` to the container `devops-container10509` to delete blobs after `7` days of last modification.

5) Validation:

- Verify that the Lifecycle Management rule named `devops-del-rule` is correctly applied.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
```

```
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Storage Account

```
az storage account create --name "devopsstor10509" --resource-group "$RG"
--location eastus --sku Standard_LRS
```

Create the Blob Container

```
az storage container create --name "devops-container10509" --account-name
"devopsstor10509" --auth-mode login
```

Upload the File

```
az storage blob upload --account-name "devopsstor10509" --container-name
"devops-container10509" --name "tempfile.txt" --file "/root/tempfile.txt"
--auth-mode login
```

Configure Blob Lifecycle Management

Lifecycle management policies are defined using a JSON structure. This rule targets the specific container and deletes blobs 7 days after their last modification.

Create the file name **lifecycle_policy.json** in Client Host

```
touch lifecycle_policy.json
```

Add the following in the file

```
{
  "rules": [
    {
      "enabled": true,
      "name": "devops-del-rule",
      "type": "Lifecycle",
      "definition": {
        "actions": {
          "baseBlob": {
            "delete": { "daysAfterModificationGreaterThan": 7 }
          }
        },
        "filters": {
          "blobTypes": [ "blockBlob" ],
          "prefixMatch": [ "devops-container10509/" ]
        }
      }
    }
  ]
}
```

Apply the policy to the storage account

```
az storage account management-policy create --account-name
"devopsstor10509" --resource-group "$RG" --policy @lifecycle_policy.json
```

Verify

To verify the rule is correctly applied, we can retrieve the policy and check for the rule name.

```
az storage account management-policy show --account-name "devopsstor10509"
--resource-group "$RG" --query "policy.rules[?name=='devops-del-rule']"
--output table
```

Verify via Azure Console

- Navigate to the **Azure Portal**
- Search for and select **Storage accounts**.
- Click on your account: **devopsstor10509**.
- In the left-hand sidebar, scroll down to the **Data management** section.
- Select **Lifecycle management**.
- We will see a list of rules. Click on **devops-del-rule** to view its specific details.

Day 37: Setting Up MySQL on a Virtual Machine in Azure

Task Description:

The Nautilus DevOps team is tasked with integrating a PHP application hosted on an Azure VM with a MySQL database hosted on another Azure VM. This will validate the application's ability to connect to the database in the cloud.

1. **Create the MySQL VM**
   - Create a VM named `xfusion-mysql-vm` using the **MySQL Jetware** image from the Azure Marketplace.
   - Configure the VM in the **Central US** region.
   - Use `Password` as the authentication type.
   - Set the **username** as `xfusion_admin` and the **password** as `Namin@123456`.
   - Allow inbound traffic on port `3306` to enable MySQL access.

2. **Setup the MySQL Database**:
   - SSH into the `xfusion-mysql-vm`.
   - Use the `sudo /jet/enter mysql` command to access the MySQL shell.
   - Create a database named `xfusion_db`.
   - Create a MySQL user named `xfusion_user` with password `password123`.
   - Grant all privileges on the `xfusion_db` database to this user.

3. **PHP VM Setup**:
   - A VM named `xfusion-php-vm` already exists in the **East US** region.
   - This VM is hosting a PHP application and contains a pre-existing `db_test.php` file in the `/var/www/html/` directory.

4. **Database Connection Configuration**:
   - Retrieve the **public IP address** of the `xfusion-mysql-vm`.
   - Update the database connection settings in the `db_test.php` file to use the MySQL credentials and public IP address of the `xfusion-mysql-vm`.

5. **Validation**:
   - Access the `db_test.php` file from the `xfusion-php-vm` using its public IP address.
   - Ensure the file displays the message `Connected successfully`, confirming the connection between the PHP application and the MySQL database.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the MySQL VM

Find the Correct Publisher and URN

```
az vm image list-publishers -l centralus --query "[?contains(name,
'jetware')].name" -o tsv
```

Once we have the publisher name (likely `jetware-srl`), we can find the specific URN for the MySQL stack:
Use the publisher name found above

```
az vm image list --all --publisher "jetware-srl" --offer "mysql" --output
```

```
table
```

use **URN** in the image parameter

Create the VM

```
az vm create --resource-group "$RG" --name "xfusion-mysql-vm" --location
centralus --image "jetware-srl:mysql:mysql57-ubuntu-1604:1.0.170503"
--admin-username "xfusion_admin" --admin-password "Namin@123456"
--authentication-type password --public-ip-sku Standard --storage-sku
"Standard_LRS" --size "Standard_B2s"
```

Open MySQL port 3306

```
az vm open-port --resource-group "$RG" --name "xfusion-mysql-vm" --port
3306 --priority 1001
```

Setup the MySQL Database

Retrieve the public IP of our new MySQL VM and SSH into it to configure the database and users.

Get the MySQL Public IP

```
MYSQL_IP=$(az vm show -d -g "$RG" -n "xfusion-mysql-vm" --query publicIps
-o tsv)
```

SSH into the VM

```
ssh xfusion_admin@$MYSQL_IP
```

Inside the VM, run these commands

Access the MySQL shell

```
sudo /jet/enter mysql
```

Create database and user

```
CREATE DATABASE xfusion_db;
CREATE USER 'xfusion_user'@'%' IDENTIFIED BY 'password123';
GRANT ALL PRIVILEGES ON xfusion_db.* TO 'xfusion_user'@'%';
FLUSH PRIVILEGES;
EXIT;
```

Update PHP Application

Now, configure the pre-existing PHP file on the `xfusion-php-vm` to point to our new database server.

Get the PHP VM Public IP

```
PHP_IP=$(az vm show -d -g "$RG" -n "xfusion-php-vm" --query publicIps -o
tsv)
```

Use SSH to update the **db_test.php** file remotely

```
ssh azureuser@$PHP_IP
```

```
sudo sed -i "s/<mysql-vm-public-ip>/$MYSQL_IP/g" /var/www/html/db_test.php
sudo sed -i "s/nautilus_user/xfusion_user/g" /var/www/html/db_test.php
sudo sed -i "s/nautilus_db/xfusion_db/g" /var/www/html/db_test.php
```

Validation

Finally, verify the connection by requesting the PHP file via `curl`

Run the below command in the PHP VM

```
curl http://localhost/db_test.php
```

Day 38: Running Containers on Azure Virtual Machines

Task Description:

The Nautilus DevOps team needs to set up an Azure Virtual Machine (VM) to interact with an Azure Blob Storage container for storing and retrieving data. The team must create a private storage account, configure Blob Storage, and test the functionality.

Task:

1) **Azure Virtual Machine Setup**:
   - The VM named `datacenter-vm` already exists in the **East US** region.

2) **Create a Private Storage Account and Blob Container**:
   - Create a storage account named `datacenterstor18855` in the **East US** region with **Locally-redundant storage (LRS)**.
   - Create a private Blob container named `datacenter-container18855`.

3) **Retrieve Storage Account Key**:
   - Get the storage account's access key to configure access for the application.

4) **Create a Test File**:
   - SSH into the VM and create a file named `testfile.txt` in the `/home/azureuser` directory with content: "this is a test file".

5) **Upload the File to Blob Storage**:
   - Upload the `testfile.txt` file to the Blob container `datacenter-container18855` using the Azure CLI command:

```
 az storage blob upload --account-name datacenterstor18855 --account-key
<access-key> --container-name datacenter-container18855 --name testfile.txt
--file /home/azureuser/testfile.txt
```

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Storage Account
Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "datacenterstor18855" --resource-group
"$RG" --location "eastus" --sku "Standard_LRS" --kind "StorageV2"
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='datacenterstor18855'].name" -o tsv)
```

Create the Private Blob Container
By default, containers created via the CLI are private (no public read access)

```
az storage container create --name "datacenter-container18855"
```

```
--account-name "$ST_ACCOUNT" --auth-mode key
```

Check Private Container

```
az storage container show --name "datacenter-container18855" --account-name
"$ST_ACCOUNT" --query "{Name:name, PublicAccess:properties.publicAccess}"
-o table
```

Retrieve the Access Key

Retrieve the primary access key (key1)

```
STORAGE_KEY=$(az storage account keys list --account-name
"datacenterstor18855" --resource-group "$RG" --query "[0].value" -o tsv)

echo "Storage Key: $STORAGE_KEY"
```

Setup and Upload File from the VM

Get the VM Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "datacenter-vm" --query publicIps -o tsv)
```

SSH and Execute Commands

```
# SSH into the VM
ssh -i /root/.ssh/id_rsa azureuser@$VM_IP

# Inside the VM: Create the test file
echo "this is a test file" > /home/azureuser/testfile.txt

# Inside the VM: Upload to Blob Storage
az storage blob upload --account-name datacenterstor18855 --account-key
<access-key> --container-name datacenter-container18855 --name testfile.txt
--file /home/azureuser/testfile.txt
```

replace `<access-key>` with the key retrieved

Verify

```
az storage blob list --account-name "$ST_ACCOUNT" --container-name
"datacenter-container18855" --output table
```

Day 39: Deploying a Static Website Using Containers on Azure

Task Description:

The Nautilus DevOps team has been tasked with creating an internal information portal for public access. As part of this project, they need to host a static website on Azure using an Azure Storage account. The Storage account must be configured for public access to allow external users to access the static website directly via the Azure Storage URL.

Task Requirements:

1. Create an Azure Storage account named `devopswebst6194` in an existing resource group.
2. Configure the Storage account for static website hosting with `index.html` as the index document.
3. Allow public access to the static website so that the website is publicly accessible.
4. Upload the `index.html` file from the `/root/` directory of the Azure client host to the Storage account's `$web` container.
5. Verify that the website is accessible directly through the Azure Storage static website URL.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Storage Account
Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "devopswebst6194" --resource-group "$RG"
--location "eastus" --sku "Standard_LRS" --kind "StorageV2"
--allow-blob-public-access true
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='devopswebst6194'].name" -o tsv)
```

Enable Static Website Hosting
This command enables the static website feature and sets `index.html` as the default landing page.

```
az storage blob service-properties update --account-name "$ST_ACCOUNT"
--static-website --index-document "index.html" --auth-mode login
```

Upload the Website Content
When we enable static hosting, Azure automatically creates a container called `$web`. we must upload our `index.html` file there.

```
az storage blob upload --account-name "$ST_ACCOUNT" --container-name
"\$web" --name "index.html" --file "/root/index.html" --auth-mode login
```

Note: The backslash before `$web` is necessary in Bash to prevent the shell from treating it as a variable.

Verification and URL Retrieval

Get the unique Web Endpoint URL

```
WEB_URL=$(az storage account show --name "$ST_ACCOUNT" --resource-group
"$RG" --query "primaryEndpoints.web" -o tsv)

echo "Static Website URL: $WEB_URL"
```

Test access

```
curl -I "$WEB_URL"
```

Day 40: Managing Secrets with Azure Key Vault

Task Description:

The Nautilus DevOps team is focusing on improving their data security by using Azure Key Vault. Your task is to create a Key Vault with an RSA key and manage the encryption and decryption of a pre-existing sensitive file using this key.

Specific Requirements:

1. **Create a Key Vault**:
   - Name the Key Vault `nautilus-1608`.
   - Create the vault in the **East US** region.
   - Use the **Standard** pricing tier.
   - Set **Soft Delete retention** to **7 days**.
   - Use the **Vault access policy** permission model.
   - Configure an access policy that allows **Get, List, Encrypt, and Decrypt** permissions for the lab identity.

2. **Create an RSA Key**:
   - Create a key named `nautilus-key` within the Key Vault.
   - Key type: **RSA**.
   - RSA key size: **4096**.
   - Leave all other settings as default.

3. **Encrypt the Sensitive Data**:
   - Use the key to encrypt the provided `SensitiveData.txt` file (located in `/root/`) on the `azure-client` host.
   - Use the `RSA-OAEP` algorithm.
   - Base64 encode the plaintext before encryption.
   - Save the encrypted version as `EncryptedData.bin` in the `/root/` directory.

Note: If you encounter a permissions error, retrieve the Service Principal ID using:
`az account show --query user.name -o tsv`
and grant the required Key Vault permissions.

4. **Verify Decryption**:
   - Decrypt `EncryptedData.bin`.
   - Base64 decodes the decrypted output.
   - Save the result as `DecryptedData.txt` in `/root/`.
   - Ensure the decrypted file matches the original `SensitiveData.txt`.

Ensure that the Key Vault and key are correctly configured. The validation script will test your configuration by decrypting the `EncryptedData.bin` file using the key you created.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create Key Vault

```
az keyvault create --name "nautilus-1608" --resource-group "$RG" --location
eastus --sku standard --retention-days 7 --enable-rbac-authorization false
```

Get Service Principal ID

```
USER_ID=$(az account show --query user.name -o tsv)
```

Set the Access Policy
Apply the Get, List, Encrypt, and Decrypt permissions

```
az keyvault set-policy --name "nautilus-1608" --object-id "$USER_ID"
--key-permissions get list encrypt decrypt
```

Create the RSA Key
Generate a 4096-bit RSA key named `nautilus-key` within the vault.

```
az keyvault key create --vault-name "nautilus-1608" --name "nautilus-key"
--kty RSA --size 4096
```

Encrypt the Sensitive Data
Azure Key Vault expects data to be Base64 encoded before encryption. We will use the `RSA-OAEP` algorithm.

Base64 encode the plaintext

```
base64 /root/SensitiveData.txt > /root/plaintext.b64
```

Encrypt using Key Vault

```
az keyvault key encrypt --vault-name "nautilus-1608" --name "nautilus-key"
--algorithm RSA-OAEP --value "$(cat /root/plaintext.b64)" --query result -o
tsv > /root/EncryptedData.bin
```

Verify Decryption
To ensure the process is reversible and the data remains intact, we perform the inverse operation.

Decrypt the binary data

```
az keyvault key decrypt --vault-name "nautilus-1608" --name "nautilus-key"
--algorithm RSA-OAEP --value "$(cat /root/EncryptedData.bin)" --query
result -o tsv > /root/decrypted.b64
```

Base64 decode to recover the original text

```
base64 --decode /root/decrypted.b64 > /root/DecryptedData.txt
```

Final Check: Compare files

```
diff /root/SensitiveData.txt /root/DecryptedData.txt
echo "Verification Successful: Files Match"
```

Day 41: Working with Azure Table Storage

Task Description:

The Nautilus DevOps team is developing a simple 'To-Do' application using Azure Table Storage to store and manage tasks efficiently. The team needs to create an Azure Table to hold tasks, each identified by a unique taskId. Each task will have a description and a status, which indicates the progress of the task (e.g., 'completed' or 'in-progress').

1. Create an Azure Storage Account named `devopstablest20197` with a Table Storage table called `tasks`.
2. Insert the following tasks into the table:
   - Task 1: PartitionKey: 'tasks', RowKey: '1', description: 'Learn Table Storage', status: 'completed'
   - Task 2: PartitionKey: 'tasks', RowKey: '2', description: 'Build To-Do App', status: 'in-progress'
3. Verify that Task 1 has a status of 'completed' and Task 2 has a status of 'in-progress'.

Note: Use the Azure CLI to insert these tasks into the table.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Storage Account
Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "devopstablest20197" --resource-group
"$RG" --location "eastus" --sku "Standard_LRS" --kind "StorageV2"
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='devopstablest20197'].name" -o tsv)
```

Create the Table

```
az storage table create --name "tasks" --account-name "$ST_ACCOUNT"
--auth-mode key
```

Insert the Task Entities
In Azure Table Storage, every record (entity) requires a PartitionKey and a RowKey. For this app, we use `tasks` as the partition to keep them grouped together.

Insert Task 1

```
az storage entity insert --account-name "$ST_ACCOUNT" --table-name "tasks"
--entity PartitionKey=tasks RowKey=1 description="Learn Table Storage"
status=completed
```

Insert Task 2

```
az storage entity insert --account-name "$ST_ACCOUNT" --table-name "tasks"
--entity PartitionKey=tasks RowKey=2 description="Build To-Do App"
status=in-progress
```

Verification

List all entities in the tasks table

```
az storage entity query --account-name "$ST_ACCOUNT" --table-name "tasks"
--output table
```

Day 42: Backup and Delete Azure Storage Blob Container

Task Description:

The Nautilus DevOps team is currently engaged in a cleanup process, focusing on removing unnecessary data and services from their Azure environment. As part of the migration process, several resources were created for one-time use only, necessitating a cleanup effort to optimize their Azure environment.

A private blob container named `xfusion-blob-29487` already exists in the `centralus` region under storage account `xfusionst5623`.

1. Copy the contents of `xfusion-blob-29487` blob container to the `/opt` directory on the `azure-client` host (the landing host once you load this lab).
2. Delete the blob container `xfusion-blob-29487` from the storage account.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Download the blobs to /opt

We will use the `az storage blob download-batch` command. This is more efficient than downloading files one by one.

Download all blobs from the container

```
az storage blob download-batch --account-name "xfusionst5623" --source
"xfusion-blob-29487" --destination "/opt" --auth-mode login
```

Delete the Blob Container

Once we have verified that the files are safely in `/opt`, we can proceed to delete the source container to optimize the environment.

```
az storage container delete --account-name "xfusionst5623" --name
"xfusion-blob-29487" --auth-mode login
```

Day 43: Configuring Azure VM with Application Gateway

Task Description:

The Nautilus Development Team needs to set up a new Azure Virtual Machine (VM) and configure it to run a web server. This VM should be part of an Azure Application Gateway (AGW) setup to ensure high availability and better traffic management. The task involves creating a VM, setting up an AGW, configuring a backend pool, and ensuring the web server is accessible via the AGW public IP.

Create a Network Security Group (NSG): Create an NSG named `devops-nsg` and add an inbound security rule `Allow-HTTP` to allow `TCP` traffic on port `80`.

Create a Virtual Machine: Create a VM named `devops-vm` using any available Ubuntu image. Configure the instance with the following settings:

- **Size:** Choose a lightweight VM size (e.g., `Standard_B1s`).
- **Authentication:** Use `SSH public key` authentication. (Please select `use existing public key` option, create public-key locally and paste contents of `~/.ssh/id_rsa.pub`)
- **OS Disk:** Use a `Standard HDD`.
- **Networking:** Under the Advanced section, attach an existing NSG (e.g., `devops-nsg`).

Additionally, configure the instance to run a user data script during launch that:

- Install the Nginx package.
- Start the Nginx service.

Set up an Application Gateway: Set up an Azure Application Gateway named `devops-agw` with the following:

- Create and Associate it with a **public IP address** named `devops-agw-ip`.
- Attach the **backend pool**:`devops-backendpool` to the VM `devops-vm`.
- Select a **subnet** for the Application Gateway (you can create a new one if needed).

Configure HTTP Settings: Create an HTTP setting named `devops-http-settings` on port `80`

Route Traffic: Add a **listener** named `devops-listener` and a **routing rule** named `devops-routing-rule` to route traffic from the AGW frontend to the backend pool:

- Listener: Frontend IP = public IP, Frontend port = 80, Protocol = HTTP
- Routing rule: Connects `devops-listener` to `devops-backendpool` using `devops-http-settings`.

NSG Adjustments: Make sure the **NSG** attached to the VM allows **inbound TCP traffic on port 80**, so the Nginx server running on `devops-vm` is accessible via the Application Gateway public IP.

Note: Wait for the Application Gateway resource to be fully deployed before proceeding with the next steps. Deployment may take several minutes to complete.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Network Configuration

Create the Network Security Group

```
az network nsg create --resource-group "$RG" --location "westus" --name
"devops-nsg"
```

Add HTTP Rule (Port 80)

```
az network nsg rule create --resource-group "$RG" --nsg-name "devops-nsg"
--name Allow-HTTP --priority 1000 --access Allow --direction "Inbound"
--destination-port-range 80 --protocol Tcp
```

Verify NSG

```
az network nsg rule list --resource-group "$RG" --nsg-name "devops-nsg"
--query "[].{Name:name, Port:destinationPortRange, Access:access,
Priority:priority}" -o table
```

Create the VNet and Subnets for VM & Application Gateway
Application Gateway requires a dedicated subnet

Create VNet and a dedicated subnet for the VM

```
az network vnet create --name "devops-vnet" --location "westus"
--resource-group "$RG" --address-prefix "10.0.0.0/16" --subnet-name
"devops-vm-subnet" --subnet-prefix "10.0.1.0/24"
```

Verify

```
az network vnet show --resource-group "$RG" --name "devops-vnet" --query
"{Name:name, Location:location, AddressSpace:addressSpace.addressPrefixes[0],
Status:provisioningState}" -o table
```

Create the REQUIRED dedicated subnet for the AGW

```
az network vnet subnet create --name "devops-agw-subnet" --vnet-name
"devops-vnet" --resource-group "$RG" --address-prefix "10.0.2.0/24"
```

Creation of Virtual Machine

Generate SSH Key Pair in client Host machine

```
ssh-keygen -t rsa -N ""
```

Create script file

```
touch userscript.txt
```

Add the following content in the file

```
#!/bin/bash

sudo apt-get update
sudo apt-get install -y nginx

sudo systemctl start nginx
sudo systemctl enable nginx
```

Create the VM

```
az vm create --resource-group "$RG" --location "westus" --name "devops-vm"
--image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--location "westus" --admin-username "azureuser" --size "Standard_B1s"
--ssh-key-values "~/.ssh/id_rsa.pub" --custom-data userscript.txt
--storage-sku "Standard_LRS" --vnet-name "devops-vnet" --subnet
"devops-vm-subnet" --nsg "devops-nsg" --public-ip-address ""
```

Creation of Application Gateway

Create Public IP

```
az network public-ip create --resource-group "$RG" --name "devops-agw-ip"
--location "westus" --allocation-method "Static" --sku "Standard" --version
IPv4
```

Get the IP

```
az network public-ip show --resource-group "$RG" --name "devops-agw-ip"
--query ipAddress --output tsv
```

Create the Application Gateway

```
az network application-gateway create --name "devops-agw" --resource-group
"$RG" --location "westus" --capacity 1 --sku Standard_v2
--public-ip-address "devops-agw-ip" --vnet-name "devops-vnet" --subnet
"devops-agw-subnet" --frontend-port 80 --http-settings-port 80
```

```
--http-settings-protocol Http
```

Routing Configuration

Add VM to Backend Pool

Get NIC of the VM

```
NIC_ID=$(az vm show --resource-group "$RG" --name "devops-vm" --query
"networkProfile.networkInterfaces[0].id" -o tsv)

echo $NIC_ID
```

Get the VM Private IP of the VM

```
VM_PRIVATE_IP=$(az vm list-ip-addresses -g "$RG" -n "devops-vm" --query
"[0].virtualMachine.network.privateIpAddresses[0]" -o tsv)
```

Add to backend pool

```
az network application-gateway address-pool create --gateway-name
"devops-agw" --resource-group "$RG" --name "devops-backendpool" --servers
"$VM_PRIVATE_IP"
```

Create HTTP Settings

```
az network application-gateway http-settings create --gateway-name
"devops-agw" --resource-group "$RG" --name "devops-http-settings" --port 80
--protocol Http
```

Create Listener

```
az network application-gateway http-listener create --gateway-name
"devops-agw" --resource-group "$RG" --name "devops-listener" --frontend-ip
appGatewayFrontendIP --frontend-port appGatewayFrontendPort --protocol Http
```

Create Routing Rule

```
az network application-gateway rule create --gateway-name "devops-agw"
--resource-group "$RG" --name "devops-routing-rule" --http-listener
"devops-listener" --rule-type Basic --address-pool "devops-backendpool"
--http-settings "devops-http-settings"
```

Final Validation
Once the AGW deployment finishes, retrieve the Public IP and test the connection.

Get the AGW Public IP

```
AGW_IP=$(az network public-ip show -g "$RG" -n "devops-agw-ip" --query
ipAddress -o tsv)
```

Testing Access

```
echo "Testing access at http://$AGW_IP"
curl -I "http://$AGW_IP"
```

Day 44: Integrating Azure Event Hub with Virtual Machines

Task Description:

The Nautilus DevOps team wants to integrate an Azure Virtual Machine with Azure Event Hubs for centralized log collection. Follow these steps to complete the task:

1. **Create Azure Event Hubs Namespace:**
   - Create an Event Hubs namespace named `xfusion-namespace` in the East US region.
   - Select the **Standard** pricing tier. Make sure to enable `Enable Auto-inflate`.
2. **Create an Event Hub:**
   - Within the namespace, create an Event Hub named `xfusion-hub`.
3. **Verify the Virtual Machine Configuration:**
   - A VM named `xfusion-vm` already exists.
   - A Python script named `send_logs.py` already exists on the VM under `/home/azureuser`. This script is used to send logs to the Event Hub. Make sure to execute this script mutiple times.
4. **Verify Logs:**
   - Ensure the logs are successfully sent to the Event Hub by checking the Event Hubs metrics in the Azure portal.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Event Hubs Namespace
We will create the namespace with **Auto-inflate** enabled. This feature allows the namespace to automatically scale its Throughput Units (TUs) if our log traffic exceeds the current limit.

Create the Namespace with Auto-inflate

```
az eventhubs namespace create --name "xfusion-namespace" --resource-group
"$RG" --location eastus --sku Standard --enable-auto-inflate true
--maximum-throughput-units 5
```

Create the Event Hub
Create the actual data stream (the hub) within that namespace.

```
az eventhubs eventhub create --name "xfusion-hub" --resource-group "$RG"
--namespace-name "xfusion-namespace" --partition-count 2 --cleanup-policy
Delete
```

Retrieve Connection String
The `send_logs.py` script on our VM will need a connection string to authenticate. We'll pull the primary connection string for the `RootManageSharedAccessKey`

Get the connection string

```
CONNECTION_STR=$(az eventhubs namespace authorization-rule keys list
```

```
--resource-group "$RG" --namespace-name "xfusion-namespace" --name
"RootManageSharedAccessKey" --query primaryConnectionString -o tsv)

echo "Copy this connection string for script:"
echo "$CONNECTION_STR"
```

Configure and Execute the Script on the VM
SSH into the VM. Update the `send_logs.py` script with the connection string and hub name.

SSH into the VM

```
ssh azureuser@<VM_IP>
```

Edit the script to add Connection String and Hub Name

```
nano /home/azureuser/send_logs.py
```

Execute the script multiple times

```
python3 /home/azureuser/send_logs.py
python3 /home/azureuser/send_logs.py
python3 /home/azureuser/send_logs.py
```

Verify Metrics
To ensure the logs are arriving, we can check the metrics via the Portal:

Portal Verification:
Navigate to **xfusion-namespace** > **Monitoring** > **Metric**s. Look for the **Incoming Messages** or **Incoming Requests** chart.

Day 45: Azure Kubernetes Service (AKS) Setup and Management

Task Description:

The Nautilus DevOps team is tasked with preparing an AKS cluster to deploy a Kubernetes-based application. The team has the following requirements:

1. Create an AKS cluster named `xfusion-aks`.
2. The Kubernetes version must be `1.33.0`.
3. The AKS cluster endpoint access must be private.
4. Ensure the cluster is created in the `Central US` region.
5. Edit the `agentpool` Node pools (delete all other node pool if exists) and configure the cluster with the following properties:
    - Node size: `D2s v3`.
    - Minimum node count: `1`.
    - Maximum node count: `2`.

- Disable the `Container Insights` for now and disable all kind of monitoring as well.

The AKS cluster must be configured with high availability and private endpoint access. Verify that the cluster meets the requirements and is ready for workloads.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Private AKS Cluster

```
az aks create --resource-group "$RG" --name "xfusion-aks" --location
"centralus" --kubernetes-version 1.33.0 --enable-private-cluster
--node-vm-size Standard_D2s_v3 --node-count 1 --enable-cluster-autoscaler
--min-count 1 --max-count 2 --nodepool-name agentpool --generate-ssh-keys
--disable-addons monitoring --no-wait
```

Check Cluster Status

```
az aks show --resource-group "$RG" --name "xfusion-aks" --query
"provisioningState" -o tsv
```

Disable Additional Monitoring

```
az aks update --resource-group "$RG" --name "xfusion-aks"
--disable-azure-monitor-metrics
```

Day 46: EventHub to Blob Storage Integration Setup

Task Description:

The Nautilus DevOps team wants to integrate an Azure Virtual Machine with Azure Event Hubs and Azure Blob Storage for centralized log collection and backup. Follow these steps to complete the task:

1. **Create Azure Event Hubs Namespace:**
   - Create an Event Hubs namespace named `devops-namespace` in the East US region.
   - Select the **Standard** pricing tier. Make sure to enable `Enable Auto-inflate`.
2. **Create an Event Hub:**
   - Within the namespace, create an Event Hub named `devops-hub`.
3. **Set Up Azure Blob Storage for Log Backup:**
   - Create a Storage Account named `devopsst11525` in the **East US** region.
   - Create a container named `devops-backup-873` within the Storage Account.
   - Ensure the container is publicly accessible for read operations.
4. **Verify the Virtual Machine Configuration:**
   - The **client host** already has a Python script named `send_logs.py` located under `/root`. This script is used to send logs to the Event Hub.
   - **Create a Virtual Machine** named `devops-vm` in the **East US** region.
   - Copy the `send_logs.py` script from the **client host** to the `/home/azureuser` directory of the `devops-vm`.
   - Modify the script on the VM to also back up the logs to the `devops-backup-873` container in the Azure Blob Storage account.
5. **Verify Logs:**
   - Ensure the logs are successfully sent to the Event Hub by checking the Event Hubs metrics in the Azure portal.
   - Verify that the logs are backed up to the `devops-backup-873` container in the Azure Blob Storage.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Event Hubs Namespace
We will create the namespace with **Auto-inflate** enabled. This feature allows the namespace to automatically scale its Throughput Units (TUs) if our log traffic exceeds the current limit.

Create the Namespace with Auto-inflate

```
az eventhubs namespace create --name "devops-namespace" --resource-group
"$RG" --location "eastus" --sku Standard --enable-auto-inflate true
--maximum-throughput-units 5
```

Create the Event Hub
Create the actual data stream (the hub) within that namespace.

```
az eventhubs eventhub create --name "devops-hub" --resource-group "$RG"
--namespace-name "devops-namespace" --partition-count 2 --cleanup-policy
Delete
```

Retrieve Connection String
We'll pull the primary connection string for the `RootManageSharedAccessKey`

Get the connection string

```
CONNECTION_STR=$(az eventhubs namespace authorization-rule keys list
--resource-group "$RG" --namespace-name "devops-namespace" --name
"RootManageSharedAccessKey" --query primaryConnectionString -o tsv)

echo "Copy this connection string for script:"
echo "$CONNECTION_STR"
```

Create the Storage Account
Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "devopsst11525" --resource-group "$RG"
--location "eastus" --sku "Standard_LRS" --kind "StorageV2"
--allow-blob-public-access true
```

The `--allow-blob-public-access` flag is the crucial part here. Without it, we cannot make the container public.

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='devopsst11525'].name" -o tsv)
```

Create the Public Blob Container

```
az storage container create --name "devops-backup-873" --account-name
"$ST_ACCOUNT" --public-access container
```

Store Public Blob Container name in variable

```
PUBLIC_BLOB_CONTAINER=$(az storage container list --account-name
"$ST_ACCOUNT" --query "[?name=='devops-backup-873'].name" -o tsv)

echo "Target Public Container: $PUBLIC_BLOB_CONTAINER"
```

Retrieve blob connection String

```
STORAGE_KEY=$(az storage account keys list --account-name devopsst11525
--resource-group "$RG" --query "[0].value" -o tsv)
```

Check Container

```
az storage container show --name "$PUBLIC_BLOB_CONTAINER" --account-name
"$ST_ACCOUNT" --query "{Name:name, PublicAccess:properties.publicAccess}"
-o table
```

Command to create VM

```
az vm create --resource-group "$RG" --name "devops-vm" --location "eastus"
--image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B1s" --admin-username "azureuser" --generate-ssh-keys
--public-ip-sku Standard --nsg-rule "SSH" --storage-sku "Standard_LRS"
```

Check VM

```
az vm show --resource-group "$RG" --name "devops-vm" --show-details --query
"{Name:name, Status:powerState, Size:hardwareProfile.vmSize,
Image:storageProfile.imageReference.offer}" -o table
```

Configure the VM

Get the VM's Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "devops-vm" --query publicIps -o tsv)
```

Copy the script from the client host to the VM

```
scp /root/send_logs.py azureuser@$VM_IP:/home/azureuser/
```

Install Required Python Libraries on VM

SSH into VM

```
ssh azureuser@<VM_IP>
```

Install dependencies

```
sudo apt update
sudo apt install python3-pip -y
```

```
pip3 install azure-eventhub azure-storage-blob
```

Update the Python Script
We will need to modify `send_logs.py` on the VM. It should now include logic to upload a copy of the log to Blob Storage.
Add **Event Hub** Connection String and **Blob** Connection String in the `send_logs.py` script

Run Script

```
python3 send_logs.py
```

Verify Logs

Event Hub
Go to Azure Portal:

- Event Hubs → `devops-namespace`
- Metrics → Incoming Messages / Requests

Blob Storage

```
az storage blob list --container-name devops-backup-873 --account-name
devopsst11525 --account-key $STORAGE_KEY -o table
```

Day 47: SQL Database Migration and Setup

Task Description:

The Nautilus DevOps team is strategizing the migration of a portion of their infrastructure to Azure. Recognizing the scale of this undertaking, they have opted to approach the migration in incremental steps rather than as a single massive transition. As part of this migration, they are focusing on setting up and managing Azure SQL Databases, implementing backup processes, and ensuring data recovery. Below are the tasks they require you to perform:

Task 1: Create an Azure SQL Database

1. Create a **publicly accessible** Azure SQL Database instance with the following details:
    - **Database Name**: `xfusion-sqldb`.
    - **Server Name**: `xfusion-server-5821`.
    - **Location**: `West US`
    - **Backup Storage Redundancy**: **Locally-redundant backup storage**.
    - **Hardware Configuration**: **Basic (For less demanding workloads)**.
    - **Admin Username**: `xfusion-admin`.
    - **Admin Password**: Set an appropriate password.
    - **Database Size**: Set to **2 GiB**.
    - Keep all other configurations as `default`.
2. Ensure the database is in the `Ready` state.

Task 2: Create a Storage Account

1. Create a **Storage Account** named `xfusionst13527`.
2. Configure a **Blob Container** named `xfusion-container-32473` within this storage account.

Task 3: Backup the Azure SQL Database

1. Take a backup of the Azure SQL Database instance `xfusion-sqldb` and store it in the **Blob Container**:
    - **Storage Account**: `xfusionst13527`.
    - **Blob Container**: `xfusion-container-32473`.
    - **Backup File Name**: `xfusion-db-backup`.
2. Ensure the backup is fully exported to the blob container.

Task 4: Download the Backup

1. Download the backup file from the Blob Container to the `/opt` directory on the `azure-client` host.
2. Ensure the file is accessible and properly named based on its extension.

Requirements for Completion

- Ensure the SQL Database is in the `Ready` state.
- Confirm the backup is stored in the specified Blob Container.
- Verify the backup file is successfully downloaded to the `/opt` directory on the client host.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.

Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Task 1: Create the Azure SQL Server and Database

Create the SQL Server

```
az sql server create --name "xfusion-server-5821" --resource-group "$RG"
--location "westus" --admin-user "xfusion-admin" --admin-password
"<YourPassword123!>"
```

Note: Replace **<YourPassword123!>** with a secure password from your credentials

Allow Azure services to access the server (needed for export/import)

```
az sql server firewall-rule create --resource-group "$RG" --server
"xfusion-server-5821" -n "AllowAzure" --start-ip-address 0.0.0.0
--end-ip-address 0.0.0.0
```

Create the Database

```
az sql db create --resource-group "$RG" --server "xfusion-server-5821"
--name "xfusion-sqldb" --edition Basic --capacity 5 --max-size 2GB
--backup-storage-redundancy Local
```

Task 2: Create the Storage Account

Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "xfusionst13527" --resource-group "$RG"
--location "westus" --sku "Standard_LRS" --kind "StorageV2"
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='xfusionst13527'].name" -o tsv)
```

Create the Blob Container

```
az storage container create --name "xfusion-container-32473" --account-name
"$ST_ACCOUNT" --auth-mode login
```

Store Blob Container name in variable

```
BLOB_CONTAINER=$(az storage container list --account-name "$ST_ACCOUNT"
--query "[?name=='xfusion-container-32473'].name" -o tsv)

echo "Target Blob Container: $BLOB_CONTAINER"
```

Retrieve blob connection String

```
STORAGE_KEY=$(az storage account keys list --account-name "xfusionst13527"
--resource-group "$RG" --query "[0].value" -o tsv)
```

Check Container

```
az storage container show --name "$BLOB_CONTAINER" --account-name
"$ST_ACCOUNT" -o table
```

Task 3: Backup (Export) the Database

Azure SQL backups are exported as `.bacpac` files. We will trigger the export and wait for it to finish.

Trigger the Export

```
az sql db export --name "xfusion-sqldb" --server "xfusion-server-5821"
--resource-group "$RG" --admin-password "<YourPassword123!>" --admin-user
"xfusion-admin" --storage-key "$STORAGE_KEY" --storage-key-type
StorageAccessKey --storage-uri
"https://xfusionst13527.blob.core.windows.net/xfusion-container-32473/xfusi
on-db-backup.bacpac"
```

Note: Replace **<YourPassword123!>** with a secure password from your credentials

Task 4: Download the Backup to `/opt`

Once the export is complete, download the file to the local **azure-client** host.

Ensure `/opt` exists

```
sudo mkdir -p /opt
```

Download the file

```
az storage blob download --account-name "xfusionst13527" --container-name
"xfusion-container-32473" --name "xfusion-db-backup.bacpac" --file
"/opt/xfusion-db-backup.bacpac" --auth-mode key --account-key
"$STORAGE_KEY"
```

Day 48: VM and ACR Integration for Storage

Task Description:

The Nautilus DevOps team needs to set up an Azure Virtual Machine (VM) to interact with an Azure Blob Storage container for storing and retrieving data. The team must create a private storage account, configure Blob Storage, and test the functionality.

Task:

1) Azure Virtual Machine Setup:

- Create a VM named `nautilus-vm` in the **East US** region.
- Authentication: Use `SSH public key` authentication. (Please select `use existing public key` option, create public-key locally and paste contents of `~/.ssh/id_rsa.pub`).
- Install Docker and Azure CLI on the VM.
- Pull the Docker image from the ACR and run it on the VM, ensuring it listens on port 80.

2) Azure Container Registry (ACR) Setup:

- Create an ACR named `nautilusacr5506` in the **East US** region.
- The repository name should be `nautilus/python-app`.
- Build the Docker image using the Dockerfile already given under `/root/pyapp`.
- Push the Docker image to the ACR with the tag `latest`.

3) Create a Storage Account and Blob Container:

- Create a storage account named `nautilusstor5506` in the **East US** region with **Locally-redundant storage (LRS)**.
- Create a Blob container named `nautilus-config`.
- Upload a file named `config.json` (available under `/root/`) to the container.

4) Validation:

- Confirm that the application can be accessed on the browser.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.

Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group

List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Azure Container Registry

```
az acr create --resource-group "$RG" --name "nautilusacr5506" --sku Basic
--location "eastus"
```

Build and Push the Image

Navigate to the app directory

```
cd /root/pyapp
```

Log in to ACR

```
az acr login --name "nautilusacr5506"
```

Build the image locally

```
docker build -t nautilusacr5506.azurecr.io/nautilus/python-app:latest .
```

Push the image

```
docker push nautilusacr5506.azurecr.io/nautilus/python-app:latest
```

Verify

```
az acr repository list --name "nautilusacr5506" --output table
```

Storage Account & Config Upload

Create Storage Account
Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "nautilusstor5506" --resource-group "$RG"
--location "eastus" --sku Standard_LRS --kind "StorageV2"
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='nautilusstor5506'].name" -o tsv)
```

Create the Blob Container

```
az storage container create --name "nautilus-config" --account-name
"$ST_ACCOUNT" --auth-mode login
```

Store Blob Container name in variable

```
BLOB_CONTAINER=$(az storage container list --account-name "$ST_ACCOUNT"
--query "[?name=='nautilus-config'].name" -o tsv)

echo "Target Blob Container: $BLOB_CONTAINER"
```

Retrieve blob connection String

```
STORAGE_KEY=$(az storage account keys list --account-name "$ST_ACCOUNT"
--resource-group "$RG" --query "[0].value" -o tsv)
```

Check Container

```
az storage container show --name "$BLOB_CONTAINER" --account-name
"$ST_ACCOUNT" -o table
```

Upload config.json

```
az storage blob upload --account-name "$ST_ACCOUNT" --container-name
"$BLOB_CONTAINER" --name "config.json" --file "/root/config.json"
--auth-mode login
```

Virtual Machine Setup

We will use a `cloud-init` script to automate the installation of Docker and the Azure CLI, and to log into the ACR.

Create script file in client Host machine

```
touch userscript.txt
```

Add the following content in the file

```
#!/bin/bash

curl -fsSL https://get.docker.com -o get-docker.sh && sh get-docker.sh

curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Generate SSH Key Pair in local

```
ssh-keygen -t rsa -N ""
```

Create the VM

```
az vm create --resource-group "$RG" --name "nautilus-vm" --location
"eastus" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B2s" --admin-username "azureuser" --ssh-key-values
@~/.ssh/id_rsa.pub --public-ip-sku Standard --nsg-rule "SSH" --storage-sku
"Standard_LRS" --custom-data userscript.txt
```

Identify NSG Name

```
NSG_NAME=$(az network nsg list --resource-group "$RG" --query
"[?contains(name, 'nautilus-vm')].name" -o tsv)

echo "Found NSG: $NSG_NAME"
```

Create the Allow-HTTP Rule

```
az network nsg rule create --resource-group "$RG" --nsg-name "$NSG_NAME"
--name "Allow-HTTP" --protocol "Tcp" --direction "Inbound" --priority 1001
--source-address-prefixes "*" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 80 --access
"Allow"
```

Retrieve the Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "nautilus-vm" --query "publicIps" -o tsv)
```

Running the App on the VM

SSH into VM

```
ssh azureuser@$VM_IP
```

Check the Docker version

```
sudo docker -v
```

Check the Azure CLI version

```
az -v
```

Download the config file from Storage to the VM

```
az storage blob download --account-name "nautilusstor5506" --container-name
"nautilus-config" --name "config.json" --file "config.json" --account-key
"<STORAGE_ACCOUNT_KEY>"
```

Note: Replace the Storage Account Key

Log in to ACR using the Admin credentials

```
sudo docker login nautilusacr5506.azurecr.io -u <ACR_USER> -p <ACR_PASS>
```

Note: Replace Username and Password

Pull the image

```
sudo docker pull nautilusacr5506.azurecr.io/nautilus/python-app:latest
```

Run the Image

```
sudo docker run -d -p 80:80 -v /home/azureuser/config.json:/app/config.json
nautilusacr5506.azurecr.io/nautilus/python-app:latest
```

Validation

Browser: Open `http://<VM_IP>` in our browser. We should see the Python app's interface.

Storage: We can verify the `config.json` exists via:

```
az storage blob exists --account-name "$ST_ACCOUNT" --container-name
"$BLOB_CONTAINER" --name config.json
```

Day 49: VM Setup with Web Storage Integration

Task Description:

The Nautilus DevOps team is tasked with setting up an environment to host a static web application. The application will serve static content from an Azure Storage Account, and a Virtual Machine (VM) will be configured to fetch and display this content using Nginx. The Azure Storage Account is used as a secure, centralized location for storing the `index.html` file. The team intentionally keeps this file outside the main source code repository, since that repository contains additional internal application code that should not be exposed to or accessed by the VM. By placing only the required static file in the Storage Account, the team can distribute this asset safely and independently of the full codebase.

The VM should securely download the index.html blob directly from the designated container (e.g., using Azure CLI, SAS URL, or REST API) and place it in Nginx's web root directory so that it is served locally by Nginx. The Storage Account is not mounted, and the Static Website feature is not used. The VM retrieves the file during deployment and may re-fetch it whenever updates are needed. The resources must follow best practices for security, performance, and accessibility.

Task Details:

1) **Create a Virtual Network (VNet) and Subnet**:
- Create a VNet named `xfusion-vnet` in the **East US** region.
- Create a subnet named `xfusion-subnet` within the VNet for the VM.

2) **Create an Azure Storage Account**:
- Create a storage account named `xfusionstor27286` in the **East US** region with **Locally-redundant storage (LRS)**.
- Create a Blob container named `xfusion-container` in the storage account.
- Upload the `index.html` file located at `/root` on the **client host** to the container `xfusion-container`.
- **Ensure the Storage Account is private and not publicly accessible** by disabling public access for the storage account.

3) **Create a Virtual Machine (VM)**:
- Create a VM named `xfusion-vm` in the **East US** region.
- Use the **xfusion-vnet** and subnet **xfusion-subnet** for the VM.
- Authentication: Use `SSH public key` authentication. (Please select `use existing public key` option, create public-key locally and paste contents of `~/.ssh/id_rsa.pub`)
- Install **Nginx** on the VM.
- Download the `index.html` file using a command such as:

```
sudo az storage blob download --account-name xfusionstor27286 --account-key
xxxxx --container-name xfusion-container --name index.html --file
/var/www/html/index.html
```

- Ensure Nginx is configured to serve the file from `/var/www/html/index.html`.

4) **Verify Setup**:
- Verify that the Nginx web server on the **client host** serves the `index.html` file correctly when accessing the VM's public IP address.

Note: Follow best practices for security, accessibility, and performance while configuring resources.
Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Virtual Network (VNet) and Subnet

```
az network vnet create --resource-group "$RG" --name "xfusion-vnet"
--location "eastus" --address-prefixes "10.0.0.0/16" --subnet-name
"xfusion-subnet" --subnet-prefixes "10.0.1.0/24"
```

Check VNet

```
az network vnet show --resource-group "$RG" --name "xfusion-vnet" --query
"{Name:name, Location:location, AddressSpace:addressSpace.addressPrefixes[0],
Status:provisioningState}" -o table
```

Storage Account & Config Upload

Create Storage Account
Storage account names must be globally unique across all of Azure and consist only of lowercase letters and numbers.

```
az storage account create --name "xfusionstor27286" --resource-group "$RG"
--location "eastus" --sku Standard_LRS --kind "StorageV2"
--allow-blob-public-access false
```

Store Storage Account name in variable

```
ST_ACCOUNT=$(az storage account list --query
"[?name=='xfusionstor27286'].name" -o tsv)
```

Create the Blob Container

```
az storage container create --name "xfusion-container" --account-name
"$ST_ACCOUNT" --auth-mode login
```

Store Blob Container name in variable

```
BLOB_CONTAINER=$(az storage container list --account-name "$ST_ACCOUNT"
--query "[?name=='xfusion-container'].name" -o tsv)

echo "Target Blob Container: $BLOB_CONTAINER"
```

Retrieve blob connection String

```
STORAGE_KEY=$(az storage account keys list --account-name "$ST_ACCOUNT"
--resource-group "$RG" --query "[0].value" -o tsv)
```

Check Container

```
az storage container show --name "$BLOB_CONTAINER" --account-name
"$ST_ACCOUNT" -o table
```

Upload **index.html** file

```
az storage blob upload --account-name "$ST_ACCOUNT" --container-name
"$BLOB_CONTAINER" --name "index.html" --file "/root/index.html" --auth-mode
key --account-key "$STORAGE_KEY"
```

Virtual Machine Setup

We will use a `cloud-init` script to automate the installation of Docker and the Azure CLI, and to log into the ACR.

Create script file in client Host machine

```
touch userscript.txt
```

Add the following content in the file

```
#!/bin/bash

sudo apt-get update

# Install Nginx
sudo apt-get install -y nginx

# Start nginx
sudo systemctl start nginx
sudo systemctl enable nginx

# Install Azure CLI
curl -sL https://aka.ms/InstallAzureCLIDeb | sudo bash
```

Generate SSH Key Pair in local

```
ssh-keygen -t rsa -N ""
```

Create the VM

```
az vm create --resource-group "$RG" --name "xfusion-vm" --location "eastus"
--image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B2s" --vnet-name "xfusion-vnet" --subnet "xfusion-subnet"
--admin-username "azureuser" --ssh-key-values @~/.ssh/id_rsa.pub
--public-ip-sku Standard --nsg-rule "SSH" --storage-sku "Standard_LRS"
--custom-data userscript.txt
```

Identify NSG Name

```
NSG_NAME=$(az network nsg list --resource-group "$RG" --query
"[?contains(name, 'xfusion-vm')].name" -o tsv)

echo "Found NSG: $NSG_NAME"
```

Create the Allow-HTTP Rule

```
az network nsg rule create --resource-group "$RG" --nsg-name "$NSG_NAME"
--name "Allow-HTTP" --protocol "Tcp" --direction "Inbound" --priority 1001
--source-address-prefixes "*" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 80 --access
"Allow"
```

Retrieve the Public IP

```
VM_IP=$(az vm show -d -g "$RG" -n "xfusion-vm" --query "publicIps" -o tsv)
```

Running the App on the VM

SSH into VM

```
ssh azureuser@$VM_IP
```

Check the Azure CLI version

```
az -v
```

Download **index.html** file from Storage to the VM

```
az storage blob download --account-name "xfusionstor27286" --container-name
"xfusion-container" --name "index.html" --file "/var/www/html/index.html"
--account-key "<STORAGE_ACCOUNT_KEY>"
```

Note: Replace the Storage Account Key
Ensure correct permissions for Nginx

```
sudo chmod 644 /var/www/html/index.html
```

Verify
Access the VM's public IP address from browser or via curl on the client host:

```
curl -I http://$VM_IP
```

Day 50: VM Setup and Configuration for Azure Application Gateway

Task Description:

The Nautilus DevOps team needs to set up an Azure Application Gateway to manage traffic for a backend pool of virtual machines. The gateway will serve as a load balancer, distributing traffic across the VMs.

Task:
1) Azure Virtual Network and Subnet:

- Create a Virtual Network (VNet) named `nautilus-vnet` in the **East US** region.
- Create a Subnet named `nautilus-subnet` within the VNet for the VMs.
- Create a Subnet named `nautilus-apgw-subnet` within the VNet for the Application Gateway.

2) Azure Virtual Machines:

- Create two VMs named `nautilus-vm1` and `nautilus-vm2` in the **East US** region.
- Install Nginx on both VMs.
- Configure `index.html` on VM1 to display "Welcome to KKE Labs:Version 1".
- Configure `index.html` on VM2 to display "Welcome to KKE Labs:Version 2".

3) Azure Application Gateway:

- Create an Application Gateway named `nautilus-apgw` in the **East US** region.
- Assign the `nautilus-apgw-subnet` to the Application Gateway.
- Create a frontend IP configuration named `nautilus-apgw-ip`.
- Add the VMs `nautilus-vm1` and `nautilus-vm2` to the backend pool.
- Configure a basic routing rule to distribute traffic between the VMs.

4) Validation:

- Verify that the Application Gateway distributes traffic to both VMs.
- Ensure that accessing the Application Gateway URL displays either "Welcome to KKE Labs:Version 1" or "Welcome to KKE Labs:Version 2" depending on the load balancing.

Azure Credentials: (Run the `showcreds` command on the `azure-client` host to retrieve these credentials)

Solution:

First we need to check that terminal configured with Azure or not

```
az account show
```

This command displays our active **subscription**, **tenant ID**, and the **user/service principal** currently authenticated.

If the terminal is not configured with azure then Login.
Login to Azure

```
az login --service-principal -u <APPLICATION_ID> -p <SECRET> --tenant
<TENANT_ID>
```

Check config file

```
cat ~/.azure/config
```

Check Azure Credentials

```
cat ~/.azure/service_principal_entries.json
```

This command displays **Client ID**, **Client Secret** and **Tenant ID**.

Using Azure CLI

Identify Resource Group
List all available resource groups

```
az group list --output table
```

Store Resource Group in variable

```
RG=$(az group list --query "[0].name" -o tsv)

echo "Using Resource Group: $RG"
```

Create the Virtual Network (VNet) and Subnet

```
az network vnet create --resource-group "$RG" --name "nautilus-vnet"
--location "eastus" --address-prefixes "10.0.0.0/16" --subnet-name
"nautilus-subnet" --subnet-prefixes "10.0.1.0/24"
```

Create dedicated Subnet for the Application Gateway

```
az network vnet subnet create --name "nautilus-apgw-subnet" --vnet-name
"nautilus-vnet" --resource-group "$RG" --address-prefix "10.0.2.0/24"
```

Check VNet

```
az network vnet show --resource-group "$RG" --name "nautilus-vnet" --query
"{Name:name, Location:location, AddressSpace:addressSpace.addressPrefixes[0],
Status:provisioningState}" -o table
```

Virtual Machine Setup

We will use `cloud-init` to install Nginx and set the specific "Version" text on each server during deployment.

Generate SSH Key Pair in local

```
ssh-keygen -t rsa -N ""
```

VM1 Setup

Create script file in client Host machine for VM1

```
touch userscript-vm1.txt
```

Add the following content in the file

```
#!/bin/bash

sudo apt-get update

# Install Nginx
sudo apt-get install -y nginx

# Start nginx
sudo systemctl start nginx
sudo systemctl enable nginx

# Create index.html file
echo "Welcome to KKE Labs:Version 1" > /var/www/html/index.html

# Restart Nginx
sudo systemctl restart nginx
```

Create VM1

```
az vm create --resource-group "$RG" --name "nautilus-vm1" --location
"eastus" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B1s" --vnet-name "nautilus-vnet" --subnet
"nautilus-subnet" --admin-username "azureuser" --ssh-key-values
@~/.ssh/id_rsa.pub --custom-data "userscript-vm1.txt" --public-ip-address
"" --nsg "SSH" --storage-sku "Standard_LRS"
```

Identify NSG Name

```
VM1_NSG_NAME=$(az network nsg list --resource-group "$RG" --query
"[?contains(name, 'nautilus-vm1')].name" -o tsv)

echo "Found NSG: $VM1_NSG_NAME"
```

Create the Allow-HTTP Rule

```
az network nsg rule create --resource-group "$RG" --nsg-name
"$VM1_NSG_NAME" --name "Allow-HTTP" --protocol "Tcp" --direction "Inbound"
--priority 1001 --source-address-prefixes "*" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 80 --access
"Allow"
```

VM2 Setup

Create script file in client Host machine for VM2

```
touch userscript-vm2.txt
```

Add the following content in the file

```
#!/bin/bash

sudo apt-get update

# Install Nginx
sudo apt-get install -y nginx

# Start nginx
sudo systemctl start nginx
sudo systemctl enable nginx

# Create index.html file
echo "Welcome to KKE Labs:Version 2" > /var/www/html/index.html

# Restart Nginx
sudo systemctl restart nginx
```

Create VM2

```
az vm create --resource-group "$RG" --name "nautilus-vm2" --location
"eastus" --image
"Canonical:0001-com-ubuntu-minimal-jammy:minimal-22_04-lts-gen2:latest"
--size "Standard_B1s" --vnet-name "nautilus-vnet" --subnet
```

```
"nautilus-subnet" --admin-username "azureuser" --ssh-key-values
@~/.ssh/id_rsa.pub --custom-data "userscript-vm2.txt" --public-ip-address
"" --nsg "SSH" --storage-sku "Standard_LRS"
```

Identify NSG Name

```
VM2_NSG_NAME=$(az network nsg list --resource-group "$RG" --query
"[?contains(name, 'nautilus-vm2')].name" -o tsv)

echo "Found NSG: $VM2_NSG_NAME"
```

Create the Allow-HTTP Rule

```
az network nsg rule create --resource-group "$RG" --nsg-name
"$VM2_NSG_NAME" --name "Allow-HTTP" --protocol "Tcp" --direction "Inbound"
--priority 1001 --source-address-prefixes "*" --source-port-ranges "*"
--destination-address-prefixes "*" --destination-port-ranges 80 --access
"Allow"
```

Creation of Application Gateway

Create Public IP

```
az network public-ip create --resource-group "$RG" --name
"nautilus-apgw-ip" --location "eastus" --allocation-method "Static" --sku
"Standard" --version IPv4
```

Get the IP

```
az network public-ip show --resource-group "$RG" --name "nautilus-apgw-ip"
--query ipAddress --output tsv
```

Create the Application Gateway

```
az network application-gateway create --name "nautilus-apgw"
--resource-group "$RG" --location "eastus" --capacity 2 --sku Standard_v2
--public-ip-address "nautilus-apgw-ip" --vnet-name "nautilus-vnet" --subnet
"nautilus-apgw-subnet" --frontend-port 80 --http-settings-port 80
--http-settings-protocol Http
```

Routing Configuration

Add VMs to Backend Pool

Get the VM Private IP of the VM1

```
VM1_PRIVATE_IP=$(az vm list-ip-addresses -g "$RG" -n "nautilus-vm1" --query
"[0].virtualMachine.network.privateIpAddresses[0]" -o tsv)
```

Get the VM Private IP of the VM2

```
VM2_PRIVATE_IP=$(az vm list-ip-addresses -g "$RG" -n "nautilus-vm2" --query
"[0].virtualMachine.network.privateIpAddresses[0]" -o tsv)
```

Add VM1 to the backend pool

```
az network application-gateway address-pool create --gateway-name
"nautilus-apgw" --resource-group "$RG" --name "nautilus-backendpool"
--servers "$VM1_PRIVATE_IP"
```

Add VM2 to the backend pool

```
az network application-gateway address-pool create --gateway-name
"nautilus-apgw" --resource-group "$RG" --name "nautilus-backendpool"
--servers "$VM2_PRIVATE_IP"
```

Create HTTP Settings

```
az network application-gateway http-settings create --gateway-name
"nautilus-apgw" --resource-group "$RG" --name "nautilus-http-settings"
--port 80 --protocol Http
```

Create Listener

```
az network application-gateway http-listener create --gateway-name
"nautilus-apgw" --resource-group "$RG" --name "nautilus-listener"
--frontend-ip appGatewayFrontendIP --frontend-port appGatewayFrontendPort
--protocol Http
```

Create Routing Rule

```
az network application-gateway rule create --gateway-name "nautilus-apgw"
```

```
--resource-group "$RG" --name "nautilus-routing-rule" --http-listener
"nautilus-listener" --rule-type Basic --address-pool "nautilus-backendpool"
--http-settings "nautilus-http-settings"
```

Final Validation
Once the AGW deployment finishes, retrieve the Public IP and test the connection.

Get the AGW Public IP

```
AGW_IP=$(az network public-ip show -g "$RG" -n "nautilus-apgw-ip" --query
ipAddress -o tsv)
```

Testing Access

```
echo "Testing access at http://$AGW_IP"
curl -I "http://$AGW_IP"
```